

Agentic AI for Information Retrieval

A Multi-Agent System for Multi-Hop Retrieval with
Self-Reflective Validation

Student ID: D03000104

Università degli Studi di Napoli Federico II

Information Retrieval — A.Y. 2024/2025

Prof. Antonio Maria Rinaldi — Dr. Domenico Benfenati

IKNOS-DIETI Lab

September 6, 2026

Contents

1	Introduction to Agentic AI in Information Retrieval	1
1.1	From Reactive Generation to Agency	1
1.2	The Limits of Static Ranking	2
1.3	Problem Statement	3
1.4	Objectives and Contributions	3
1.5	Structure of This Document	5
2	Data and Preprocessing	6
2.1	Requirements for Evaluating an Agentic IR System	6
2.2	On the Datasets Listed in the Assignment Brief	6
2.3	HotpotQA	7
2.4	2WikiMultihopQA	7
2.5	Preprocessing Pipeline	8
2.6	Evaluation Sampling	9
3	Retrieval Methodologies	12
3.1	Classical Foundations	12
3.1.1	Sparse Retrieval: BM25	12
3.1.2	Dense Retrieval	12
3.1.3	Hybrid Fusion and Reranking	13
3.2	Why Single-Shot Retrieval Fails on Multi-Hop Needs	13
3.3	Agentic Retrieval	15
3.3.1	Query Decomposition and Refinement	15
3.3.2	Tool-Augmented Retrieval	16
3.3.3	Knowledge Graph Traversal	17
3.3.4	Self-Reflective Validation	17
3.4	The Role of the Language Model	18
3.5	Architectural Summary	19
4	System Implementation and Evaluation	20
4.1	System Architecture	20
4.1.1	Guarding against non-termination	20
4.2	Implementation Details	21
4.2.1	Indexing	21
4.2.2	Local inference	21
4.2.3	Structured output and its failure mode	22
4.2.4	Budget enforcement	22
4.3	Experimental Setup	22
4.4	Baselines	23
4.5	Evaluation Metrics	23
4.6	Results	25

4.7	Ablation Study	31
4.8	Error Analysis	34
4.9	Verifier Threshold Calibration	36
4.10	Discussion	37
5	Conclusions and Future Work	39
5.1	Summary of Contributions	39
5.2	Empirical Findings	39
5.3	Limitations	43
5.4	Future Work	45
5.5	Toward a Thesis Extension	46

Abstract

Most retrieval-augmented systems are pipelines: they retrieve once and generate once. That structure cannot answer a question whose evidence is distributed across documents, because no single document is rankable as the answer. This report presents an agentic alternative — four agents coordinated by an explicit state machine, covering all five tasks of the assignment brief. A Planner decomposes a question into a validated graph of dependent sub-queries; a Retrieval Agent routes each sub-query to BM25, dense, or hybrid search via a seven-rule decision table costing zero model calls; a Knowledge Graph Navigator traverses an entity graph induced from corpus text to recover bridge entities; and a Verifier scores the answer against its own citations using entailment and citation grounding. The central contribution is one backward edge: below its confidence threshold the Verifier returns a re-plan directive to the Planner rather than an answer. That edge is the difference between a pipeline and an agent, and it is isolated by ablation rather than asserted. The system is evaluated on HotpotQA and 2WikiMultihopQA against lexical, dense, hybrid, and agentic baselines, reporting agent-specific cost alongside answer quality. Everything runs locally on one consumer GPU with no paid API. The result is not the one the architecture was hoping for and is reported as measured. On answer accuracy the agentic system does not lead: exact match is 0.432 on HotpotQA and 0.224 on 2WikiMultihopQA against 0.504 and 0.264 for a self-ask baseline that decomposes but never verifies, a deficit that is significant on HotpotQA under the paired bootstrap and not on 2WikiMultihopQA. The backward edge itself is nonetheless doing measurable work: removing it, with everything else held fixed, changes HotpotQA F1 by -0.037 [$-0.065, -0.011$], a significant loss. On attribution the agentic system is not more accurate but differently accountable. Its supporting-fact F1 of 0.481 and 0.421 exceeds the strongest retrieval baseline’s 0.412 and 0.327 significantly, and it is scored on the sentences it actually cited, at a citation grounding of 0.960 and 0.976 where every baseline resolves no citation at all; the generative baselines, scored on their whole evidence pool, reach a supporting-fact F1 that is not comparable with it and is marked as such, while on the like-for-like recall of the whole pool the agentic system leads every baseline, 0.793 and 0.684 against 0.706 and 0.622. Deterministic routing, linking, and traversal displaced a measured share of the model calls the system would otherwise have made — though in the headline run the Planner’s own tool hint pre-empted the routing table on every sub-query, so six of its seven rules never executed — and the re-plan loop fired on 47.2% and 39.2% of questions. The defensible claim is about grounded, auditable answers rather than raw accuracy, and the cost of that guarantee is reported beside it.

Chapter 1

Introduction to Agentic AI in Information Retrieval

1.1 From Reactive Generation to Agency

A generative language model is a function from a prompt to a completion. Given an input it emits text and stops. It holds no goal that outlives the call, it cannot observe what its output caused, and it has no mechanism for concluding that what it produced was inadequate. That is an accurate description of a component, and it is also an accurate description of most systems currently labelled retrieval-augmented: a query is embedded, a ranked list is fetched, the list is concatenated into a prompt, and a completion is emitted. Retrieve once, generate once. The behaviour is reactive because nothing in the architecture allows it to be anything else.

Agentic artificial intelligence names the class of systems built to remove that limitation. An agentic system pursues a goal across multiple steps, selects its own actions, invokes external instruments to carry them out, observes what those instruments returned, and revises its behaviour in light of the observation [14, 10]. Four properties are load-bearing, and it is worth separating them because systems are routinely called agentic on the strength of the first three alone.

Autonomy means that the system, not the user, decides what happens next. The user supplies a goal; every intermediate decision — which sub-question to ask, which index to consult, whether the current answer is good enough to return — belongs to the system. **Goal-directed planning** means the system constructs an explicit representation of the work before doing it. A plan is an artefact rather than an emergent side effect: it can be inspected, validated, repaired, compared against a previous plan, and discarded. **Tool integration** means the model is not the answer source but the instrument selector. Facts come from retrieval indexes, graph traversals, and arithmetic; the model’s contribution is deciding which instrument fits which sub-goal and how the outputs compose. **Adaptive reasoning** means the plan is revisable after execution has begun, because an observation can invalidate it.

The fourth property is the difficult one, and it is the one most often absent. Autonomy, planning, and tool use can all be present in a system that is nevertheless a fixed pipeline whose stages happen to be language-model calls: work flows forward, each stage runs exactly once, and nothing discovered late can change anything decided early. The industry touchstones the assignment brief cites — AutoGPT and Microsoft’s Jarvis — are notable precisely for attempting the fourth property, letting a model re-enter its own planning stage on the basis of what its tools returned. The research literature shows the same trajectory. PaperQA retrieves across full-text scientific articles, judges the relevance of what it found, and synthesises an answer from the surviving evidence [3]. The Agentic Reasoning framework adds tool-using sub-agents and a mind-map agent that maintains a structured record of established relations [10]. CollEX supports curiosity-driven multimodal exploration of scientific collections [7], agentic contextual retrieval in telecommunications combines multi-source retrieval with self-reflective validation for network

decision-making [13], and multi-agent geospatial copilots decompose remote-sensing workflows across specialised agents [4]. Chapter 3 examines these approaches in detail. What matters here is the shape they share and the question they leave open: whether the feedback edge, once actually implemented, earns the compute it consumes.

1.2 The Limits of Static Ranking

Classical information retrieval is the study of a scoring function. Given a query and a corpus, rank the documents by estimated relevance and return the top k . The probabilistic relevance framework and its BM25 instantiation formalise this for lexical matching [6]; dense bi-encoders replace term overlap with proximity in a learned embedding space [11]; reciprocal rank fusion combines rankings from several such systems without requiring their scores to be commensurable [1]; and heterogeneous benchmarks such as BEIR measure how well any of it transfers to unseen domains [8]. These are genuine and cumulative advances. Every one of them improves the *ordering of a single list produced from a single query*.

Consider a question from the HotpotQA development set [12], which this report reuses as a running example:

When did the tour begin for the album Taylor Swift released on October 22, 2012?

The corpus contains the two passages that settle it. The passage titled **Red (Taylor Swift album)** states that *Red* is the fourth studio album by Taylor Swift and was released on October 22, 2012. The passage titled **The Red Tour** states that the tour was launched in support of that album and began on March 13, 2013 in Omaha, Nebraska. The gold answer is *March 13, 2013*.

Of the 66,581 passages in the processed HotpotQA corpus, exactly zero contain both *October 22, 2012* and *March 13, 2013*. This is not a statement about ranking quality. It is a statement about the corpus: there is no document for a ranker to place first, because no single document answers the question. A single-shot retriever is structurally incapable here, and so is every refinement layered on top of it. A cross-encoder reranker permutes a candidate set; it cannot insert into that set a passage whose relevance depends on a fact the query does not contain.

The lexical structure of the example sharpens the point. The bridge entity — the album title *Red*, the one term connecting the question to the answer-bearing page — does not occur in the question at all. It has to be *derived* before it can be searched for. A lexical retriever cannot match a term that was never typed, and a dense retriever embeds the question as a whole, so the sub-need “identify the album” is diluted by the sub-need “find its tour date” rather than issued as a query in its own right. The information need is a composition of two atomic needs, and single-shot retrieval resolves compositions only in the fortunate case where some passage already states the composition. This is the compositionality gap that Press et al. measure directly [5], and it is what motivates interleaving retrieval with reasoning steps rather than separating them [9].

Scale turns the difficulty into an impossibility. The two corpora used in this work hold 66,581 and 54,957 passages; the number of ordered passage *pairs* is on the order of 10^9 in each case. Materialising the compositions in advance is not an option, so the composition must be inferred from the question, which is a reasoning operation rather than a ranking one. Two further pressures compound it. The HotpotQA distractor setting populates each question’s candidate pool with passages deliberately chosen to be topically adjacent to the gold ones, so surface similarity is actively adversarial. And relevance for the first hop is defined only relative to an intermediate goal the user never stated: **Red (Taylor Swift album)** is relevant not because it resembles the question but because it supplies the term the second query needs. No relevance judgement over the original query alone can express that.

1.3 Problem Statement

The problem this project addresses is the design of an information retrieval agent that, given a natural-language question and a static corpus, does four things a ranker does not. It must *interpret the intent* behind a question whose surface form under-specifies the retrieval work required. It must *decompose* that intent into an ordered set of sub-goals with explicit dependencies, so that the result of one retrieval can be substituted into the query of the next. It must *orchestrate* the resulting multi-step workflow, choosing an appropriate retrieval instrument for each sub-goal and, where the corpus admits a structured view, traversing that structure rather than searching it. And it must *judge its own answer* against the evidence it actually retrieved, before returning anything.

The fourth requirement is where a pipeline and an agent part company, and it is the central claim this report exists to test. The system described here is, with one exception, a forward pipeline: plan, execute, aggregate, synthesise. The exception is a single backward edge. A Verifier scores each candidate answer against the evidence cited in support of it, blending natural-language-inference entailment, citation grounding, and the agreement between retrieved passages and the answer into one confidence value. When that confidence falls below the configured threshold, the Verifier does not answer. It emits a **ReplanDirective** back to the Planner carrying what could not be established, which sub-queries returned nothing useful, and — the field that makes the loop converge rather than spin — the normalised text of every sub-query already attempted, which the next plan is forbidden to repeat. One edge is a modest architectural change and a categorical behavioural one: it is the only mechanism in the system by which a late observation can revise an early decision.

The system operates under a deliberate constraint. It runs entirely on local hardware: inference is served by **qwen3:8b** at Q4_K_M quantisation through Ollama on a single RTX 5060 with 8 GB of VRAM, measured at 52 tokens per second warm, and no component holds an API key. The constraint sharpens the research question rather than weakening it. At that throughput a schema-constrained model call costs several seconds of wall clock, so every call must be justified against a deterministic alternative. The resulting design treats “where should an agent reason?” as a budgeting problem: tool selection is decided by a seven-rule decision table over cheap lexical features and costs zero model calls, entity linking is alias matching, graph traversal is bidirectional breadth-first search, and the confidence blend is arithmetic. The worst case for a full agentic question is 15 model calls against a hard cap of 20, and the typical cost is four to seven. A system that cannot afford to think everywhere is forced to be explicit about where thinking pays — and, as Chapter 4 reports, the division of labour it settles on is not always the one that was designed: in the headline run the Planner attached a tool hint to every sub-query, so the first rule of the routing table decided every routing and the other six were never reached.

1.4 Objectives and Contributions

The objective is a working agentic retrieval system that covers all five tasks of Table 1 of the assignment brief and whose central architectural claim is tested rather than asserted. The contributions are four.

First, a four-agent architecture with full task coverage. A Planner performs autonomous query refinement and multi-step retrieval planning, emitting a validated directed acyclic graph of sub-queries linked by inter-node placeholders. A Retrieval Agent performs tool-augmented retrieval, selecting among BM25, dense, and hybrid search under a conditional reranking gate. A Knowledge Graph Navigator performs graph traversal over an entity graph built from corpus text alone, using bidirectional search to surface the bridge entity that multi-hop questions turn on. A Verifier performs self-reflective validation and fact checking. An orchestrator, specified as an explicit state machine rather than as implicit control flow, coordinates the

four.

Second, the Verifier-to-Planner feedback loop, isolated by ablation. The loop is the difference between this system and the retrieve-once pipeline it is measured against, so it is evaluated as a treatment rather than described as a feature. An `agentic_no_verifier` configuration disables the backward edge while holding everything else fixed, and the trace records which cycle produced the answer that was ultimately selected, so the question “did re-planning actually help?” is answerable from the logs rather than inferred from an aggregate.

Third, an evaluation protocol that prices agency. Answer quality and retrieval quality are reported alongside agent-specific cost — model calls per question, calls avoided by deterministic rules, tool invocations, plan depth, re-plan rate, and latency — so that no accuracy gain is presented without the compute it consumed. The evaluation grid comprises nine configurations — lexical, dense, hybrid, naive-RAG and self-ask baselines, the agentic system and its three ablations — over two multi-hop benchmarks. At the time of writing all nine have been run on HotpotQA, one of them still on a partial slice, and six on 2WikiMultihopQA; Chapter 4 states precisely which rows are complete, and no conclusion is drawn from a partial one.

Fourth, a fully local and reproducible implementation. Every model runs on-device, prompts are versioned files whose rendered SHA-1 is logged with each call, every sort carries an explicit tiebreaker, and a test asserts that no module imports a network client other than the local Ollama endpoint. The system can be re-run by a reader with a consumer GPU and no budget.

Table 1.1 records where each of the brief’s five tasks is implemented and where this report treats it, so that the coverage claim above can be checked line by line rather than taken from the prose. One clause of the brief is deliberately not met. Its description of the implementation chapter mentions “orchestration of external APIs for real-time data enrichment”, and its Table 1 gives web search APIs as an example of a tool. This system exposes no external API at all: every tool in the registry is a local index or the local entity graph, and the network test above forbids anything else. The deviation is a design decision rather than an omission. A tool whose output changes between runs makes the evaluation unrepeatable and the trace unfalsifiable, and the constraint of running with no paid service is one the report treats as a feature of the study. The tool registry is written so that a real-time connector could be added as one more entry, and Section 5.5 returns to that extension.

Table 1.1: The five tasks of the assignment brief’s Table 1, the component that implements each, and where the report specifies and measures it. Five tasks are covered by four agents because query refinement and retrieval planning are the same act performed at two points in the loop: once before any evidence exists and again after the Verifier has said what is missing.

Task (brief, Table 1)	Agent or component	Specified in; measured in
Autonomous query refinement	Planner: per-node query rewrites; re-plan on a <code>ReplanDirective</code>	§3.3.1, §3.3.4; re-plan rate in Table 4.5
Multi-step retrieval planning	Planner: validated sub-query DAG with placeholders; orchestrator state machine	§3.3.1, §4.1; §4.7 (<code>agentic_no_planner</code>)
Tool-augmented retrieval	Retrieval Agent: tool registry, routing table, conditional reranking	§3.3.2; tool calls and calls saved in Table 4.5
Knowledge graph traversal	KG Navigator: alias linking, bidirectional BFS over the induced entity graph	§3.3.3, §2.5; §4.7 (<code>agentic_no_kg</code>)
Self-reflective validation and fact checking	Verifier: NLI entailment, citation grounding, confidence blend, feedback edge	§3.3.4; §4.7 (<code>agentic_no_verifier</code>), §4.8, Table 4.11

A fifth outcome is not a designed contribution but is reported because it is useful. Schema-constrained planning with an 8B model proved more reliable than expected on structure and less reliable than expected on self-description: across the planning calls measured during development the model produced correctly shaped decompositions in 8 of 8 cases with zero format retries, while mislabelling its own plan’s **strategy** field in 7 of those 8, including obvious bridge and comparison cases. The system therefore derives the strategy from the shape of the validated plan graph and overwrites whatever the model claimed. Where a small model’s self-report can and cannot be trusted is a practical result for anyone building agents at this scale.

1.5 Structure of This Document

Chapter 2 covers data and preprocessing. It explains why the pretraining corpora listed in the brief’s Table 2 cannot support an evaluation requiring relevance judgements, substitutes two multi-hop retrieval benchmarks that carry the necessary ground truth, and documents the passage segmentation, deduplication, and stratified sampling that produce the evaluation and calibration splits.

Chapter 3 surveys retrieval methodologies. It develops the classical foundations — sparse, dense, and fused retrieval with reranking — establishes why single-shot retrieval fails on compound needs, and then examines the agentic alternatives corresponding to each of the brief’s five tasks, locating this system’s design choices within that space.

Chapter 4 presents the implementation and the evaluation. It specifies the orchestrator state machine, the per-agent contracts, the indexing and budget-enforcement machinery, the baseline and ablation configurations, and the metrics; it then reports results, ablations, and a qualitative error analysis.

Chapter 5 assesses what the evidence supports, states the limitations of the study without softening them, and sets out the work that follows from it.

Chapter 2

Data and Preprocessing

2.1 Requirements for Evaluating an Agentic IR System

A dataset is not chosen because it is popular; it is chosen because it can answer the questions a report has to ask. This system’s central claim — that a validation agent can send useful feedback to a planning agent — is only meaningful if it can be falsified with numbers. Four requirements follow, and they are stated before any dataset is named so that the selection reads as derived rather than preferred.

(a) Queries with relevance judgments. Chapter 4 is required to report relevance metrics against baseline IR systems. Recall@ k , nDCG@10 and MRR are all defined over a mapping from queries to relevant documents. A corpus without such a mapping — a *qrels* file, in TREC terminology — makes every one of those metrics undefined, not merely inconvenient to compute. This requirement is therefore non-negotiable, and it eliminates more candidate resources than the other three combined.

(b) Genuinely compositional information needs. The Planner exists to decompose a question into a dependency graph of sub-questions. On a corpus of single-fact look-ups the correct decomposition is the identity, every plan has one node, and the entire agentic apparatus collapses onto the hybrid baseline it is supposed to beat. Measuring a planner requires questions whose answers are not contained in any single document.

(c) Entity and structure annotations. The KG Navigator traverses an entity graph, and a traversal that cannot be scored is a description rather than a result. Sentence-level supporting-fact annotations make retrieved evidence checkable at the granularity the system actually emits; gold relational triples make the traversal itself checkable.

(d) A corpus that indexes and runs on consumer hardware. Every component of this system — generation, embedding, reranking, entailment — runs locally on a single 8 GB laptop GPU, with no paid API anywhere in the project. A web-scale corpus is not merely expensive under that constraint; it is unusable, and a resource that cannot be indexed cannot be evaluated.

BEIR [8] is the natural first place to look, and it satisfies (a) and (d) comfortably across eighteen heterogeneous retrieval tasks. Most of its tasks, however, express single-hop needs and carry no reasoning-structure annotations, so they satisfy neither (b) nor (c). The two datasets adopted below satisfy all four, and the first of them is itself one of BEIR’s tasks, which keeps this choice inside the standard IR evaluation canon rather than outside it.

2.2 On the Datasets Listed in the Assignment Brief

Table 2 of the assignment brief lists five resources: The Pile, Common Crawl, WikiText, Open-WebText and LAION-5B. All five are pretraining corpora. They supply unlabelled text — or, for LAION-5B, weakly labelled image–text pairs — at a scale suitable for fitting the parameters of a language or multimodal model. None of them ships a query set, a relevance judgment, or a gold

answer, because none of them was assembled for that purpose. Requirement (a) therefore fails outright: with no query-to-document relevance mapping there is no $\text{Recall}@k$ and no $\text{nDCG}@10$ to report, and the comparison against baseline IR systems that Chapter 4 is required to present is not computable at all. This is not a deficiency in those corpora. It is a statement about what they are for.

They are, in fact, exactly the right resources for a different layer of this system. The reasoning component used throughout this project is a locally served 8B-parameter language model, and its parametric knowledge — its grasp of English syntax, of world entities, of the shape of a well-formed JSON object — is the product of pretraining on corpora of precisely the kind Table 2 enumerates. The brief’s list describes the substrate the agent brings to the task; it does not describe the benchmark against which retrieval is scored. Those are two different roles, and conflating them is what makes the substitution below look like a departure when it is not one. The brief’s own instruction to “always check for eventual updates of links and datasets via a web search” is the licence to select evaluation data on evaluation criteria, and that is what Sections 2.3 and 2.4 do.

2.3 HotpotQA

HotpotQA [12] is a multi-hop question answering dataset built over English Wikipedia, in which each question is written so that answering it requires combining facts from two distinct paragraphs. This work uses the **distractor** setting, in which every question is accompanied by its two gold paragraphs and eight retrieved distractors, and the **validation** split, which contains 7,405 questions.

Its distinguishing property, and the reason it is the primary dataset here, is the *supporting facts* annotation: alongside the answer string, every question carries a list of (**title**, **sent_id**) pairs naming the individual sentences a human annotator used. This is retrieval ground truth at sentence granularity rather than merely answer ground truth, and it is what makes the supporting-fact metrics **sp_em** and **sp_f1** computable at all. It also fixes the granularity of the rest of the system: evidence is carried as sentences, not passages, so that what the Verifier reasons over and what the metrics score are the same objects.

Reading the distractor contexts of all 7,405 validation questions yields 73,700 paragraphs, which reduce to 66,581 distinct passages after the deduplication described in Section 2.5; 7,119 of the discarded paragraphs were byte-identical repeats, and no title in this split carries two different body texts. The corpus contains 269,602 sentences, a mean of 4.049 sentences and 531.9 characters per passage. Of the 18,005 gold supporting facts, 18,004 resolve against the built corpus. The single exception is a genuine upstream annotation error rather than a preprocessing artefact: question **5ae61bfd5542992663a4f261** cites sentence 902 of the five-sentence paragraph “Jimmy Butler (basketball)”. At a rate of 5.6×10^{-5} this is noise, but it is reported rather than silently dropped, because the same counter is the tripwire that would catch a genuine sentence-identifier drift.

One measured property of this split governs the sampling design in Section 2.6 and is easy to get wrong. HotpotQA labels questions **easy**, **medium** or **hard**, but **all 7,405 validation questions carry level=hard** — the easier levels exist only in the training split. The informative partition here is the reasoning type: 5,918 **bridge** questions, in which one entity must be resolved before the second hop can even be posed, and 1,487 **comparison** questions, in which two independent entities are retrieved and an attribute of each is compared.

2.4 2WikiMultihopQA

2WikiMultihopQA [2] was constructed to expose the reasoning steps a multi-hop question requires, rather than only its endpoints. Its validation split contains 12,576 questions whose 125,760

context paragraphs reduce to 54,957 distinct passages and 202,759 sentences, a mean of 3.689 sentences and 427.2 characters per passage. Like HotpotQA it ships sentence-level supporting facts — 30,687 of them, all of which resolve.

The reason it is in this report is what it ships *in addition*: gold Wikidata evidence triples, 31,120 of them, covering all 12,576 questions. These give the reasoning chain as structured relations rather than as prose, and they convert the KG Navigator from a component this report describes into a component this report measures. They also introduce the most severe leakage risk in the project. Building the entity graph from those triples would let the KG Navigator read the answer key, and every graph-attributed gain in Chapter 4 would be an artefact of that leak. The graph is therefore built from corpus passage text only (Section 2.5); the gold triples are loaded exclusively by the scoring code, and a test asserts that the graph builder never opens a file whose name contains `evidence` or `triple`.

Unlike HotpotQA, all four of 2Wiki’s reasoning types are populated: `compositional` (5,236), `comparison` (3,040), `bridge_comparison` (2,751) and `inference` (1,549). The third of these is the hardest case in either dataset, since it requires both a bridge resolution and a comparison over its results.

This split is also textually messier than HotpotQA, in a way that bears directly on ground truth. Because the same article appears in the contexts of many questions, and because those contexts were detokenised inconsistently, 1,720 of the 54,957 titles ship two or more variants of the same article (“Feo(c. 1471” versus “Feo (c. 1471”), producing 2,617 title collisions. Since `sent_id` indexes into a sentence list, the variant retained decides whether a gold fact lands on the right sentence. The resolution rule and its measured cost are given in Section 2.5.

2.5 Preprocessing Pipeline

The pipeline is deliberately thin. Every transformation it applies is one that cannot be undone downstream, and every transformation it declines to apply is one a consumer can perform for itself.

Passage segmentation. Both datasets already ship each context paragraph as a list of sentences, and that list is adopted verbatim. Nothing in this project calls a sentence splitter — not NLTK, not PySBD, not a regular expression. The reason is that supporting facts are (`title`, `sent_id`) pairs, so the sentence list *is* the identifier space of the ground truth. Re-splitting anywhere downstream would renumber every gold fact while `sp_em` and `sp_f1` continued to produce entirely plausible numbers — a silent failure, which is the worst kind. The split is performed once, at corpus build time, and persisted. Empty sentences keep their slot, because the slot is the identifier.

Canonical identifiers. Each passage is keyed by `doc_id = {source}:{quote(title)}`, for example `hotpotqa:Accolade%20Wines`. The identifier is URL-safe, stable across rebuilds, and derived from the title alone, so that a passage returned by any of the three scorers, an entity node in the graph, and a gold supporting fact all name the same object in the same way.

Deduplication. The distractor setting repeats paragraphs across questions, so the raw paragraph stream is roughly an order of magnitude larger than the distinct corpus. HotpotQA collapses cleanly: 7,119 byte-identical repeats removed, zero collisions. 2Wiki removes 68,186 identical repeats but leaves 2,617 cases in which one title carries two different texts. The rule is to keep the variant with the *most sentences* — the widest identifier space — with ties going to the first occurrence. The rule is decided without consulting any gold data, and its cost is measured: over 2Wiki’s 30,687 gold facts, keeping the first occurrence would leave 70 identifiers out of range and 211 pointing at the wrong sentence, whereas keeping the widest leaves 0 out of

range and 150, or 0.489%, landing on a slightly different detokenisation of the intended sentence. Both residuals are counted in the build statistics rather than rounded away.

Normalisation, and what is deliberately not done. The only transformation applied to corpus text is the collapse of whitespace runs inside a sentence, which is necessary because HotpotQA prefixes continuation sentences with a space and 2Wiki does not. There is no lowercasing, no stopword removal, no stemming and no punctuation stripping at the corpus level. Those are properties of a *scorer’s analyser*, not of a corpus: BM25 applies Snowball stemming and an English stopword list inside its own analyser, whereas the dense encoder needs original case and punctuation to distinguish proper nouns, and the NLI premise encoder needs well-formed sentences. Baking any of those steps into the stored text would be irreversible and would degrade two of the three scorers in order to help one. Conversely, one convention *is* shared across all of them: every scorer sees the same title-prefixed unit, "**Title. body**". A channel that indexes titles, fused with a channel that does not, produces rankings that are not comparable, and rank fusion would absorb the mismatch silently rather than expose it.

Entity extraction and alias resolution. Graph nodes are passage titles, normalised by NFKD decomposition, lowercasing, diacritic stripping, removal of parenthetical disambiguators and whitespace collapse. A node’s alias set is its surface title plus the no-parenthetical variant plus the comma-inverted variant, and the aliases are compiled into a trie for longest-match linking at query time. No named-entity recogniser and no language model is involved, so linking is deterministic and costs nothing at run time.

Entity graph construction. A directed edge $A \rightarrow B$ is created when an alias of B occurs in the text of passage A . Each edge stores the `doc_id`, the `sent_id` and the containing sentence, which is what makes a traversal result citable evidence at the same granularity as a retrieved passage rather than an unscorable decoration. The relation label is the shortest verb-anchored token window between the two mentions, truncated to six tokens and defaulting to `mentions`; there is no parser dependency and no generation step. Edge weight is $1/(1 + \log(1 + \text{outdeg}(A)))$, so hub pages contribute weak edges. As recorded in `data/indexes/{dataset}_kg_meta.json`, the HotpotQA graph retains 64,733 nodes and 123,759 edges from 417,881 candidates and builds in 49.4s; the 2Wiki graph retains 52,959 nodes and 63,073 edges from 161,572 candidates and builds in 32.9s. Pruning is dominated by a document-frequency cap on mentions, which removed 446 and 172 hub nodes respectively: strings such as `one` and `united states` are legitimate page titles, but their occurrences are not relational assertions, and without the cap the graph collapses into a hairball centred on them. Verb-anchored relation labels are recovered for 76.8% of HotpotQA edges and 79.0% of 2Wiki edges, and Section 3.3.3 is explicit about what the remainder costs.

Indexing. Each corpus is indexed twice: a BM25 index with Snowball stemming, and an exact inner-product FAISS index over 384-dimensional `bge-small-en-v1.5` embeddings. Together these occupy 129.1 MB for HotpotQA and 103.6 MB for 2Wiki — small enough to hold in memory, which is why exact search is used in place of an approximate index and why no recall loss is introduced at the index level. The 2Wiki build took 4.1 s for BM25 and 97.6 s for the dense index on GPU.

2.6 Evaluation Sampling

Evaluation is run on a frozen 250-question sample per dataset rather than on the full split, and the reason is arithmetic. Local generation runs at a measured 52 tokens per second warm on an RTX 5060 with 8 GB of VRAM, giving a derived rather than measured planner call of

about 3.75 s. The pre-run estimate on which the sampling decision was taken put a full agentic question at roughly 22–25 s once retrieval, reranking and entailment were included; the medians actually measured, reported in Table 4.5 and Table 4.6, came out higher, and the estimate is recorded here as the estimate it was rather than revised to match. On that estimate one agentic configuration over HotpotQA’s full 7,405-question validation split is on the order of 45 hours, and the evaluation grid comprises nine configurations across two datasets. At 250 questions the same grid was estimated at 12–16 hours, which is feasible on this hardware; at 7,405 it is not.

The sample is drawn once, by `scripts/sample_eval_set.py` with seed 42, materialised to `data/processed/{dataset}_eval_250.jsonl`, and never re-sampled at run time. Its SHA-256 is recorded in every run’s `meta.json`, so a silently changed sample is detectable after the fact.

The stratification variable is question type, not difficulty, and that is a measured decision rather than a stylistic one. As reported in Section 2.3, all 7,405 HotpotQA validation questions are labelled `hard`. Stratifying on `level` would therefore produce a single stratum — an unstratified sample presented as a stratified one, with the label doing no work while implying that it does. Stratifying on `type` preserves the 0.799/0.201 bridge-comparison ratio exactly, at 200 and 50 questions. For 2Wiki the four reasoning types are preserved at 104 compositional, 60 comparison, 55 bridge-comparison and 31 inference. Table 2.2 gives the full composition against the split.

Two hundred and fifty questions is enough to separate the systems compared here. In the worst case — a metric near 0.5, treated as an unpaired binomial proportion — the 95% normal-approximation half-width is $1.96\sqrt{0.25/250} \approx 0.062$, an interval of roughly ± 6.2 points. The comparisons that matter in Chapter 4 are paired, however: every configuration answers the identical 250 questions, so the bootstrap is taken over per-question *differences* (1,000 resamples, `evaluation.bootstrap_samples`), and the error correlation between systems sharing a corpus, an index and a question set makes those intervals materially tighter than the unpaired figure suggests; the realised widths are reported with the results rather than assumed here. The three component ablations run on the same 250 questions as every other configuration, not on a subsample, so that every ablation delta is a paired comparison over an identical question set; where an ablation run had not yet covered all 250 questions the generated table marks the row as preliminary and withholds its delta rather than pairing a prefix against the full slice.

Each dataset also gets a 50-question calibration slice, `{dataset}_calib_50.jsonl`, written beside the evaluation file in `data/processed/` and verified to have zero question-identifier overlap with the evaluation 250. Its intended purpose was to fix the Verifier’s `confidence_threshold` — 0.55 — before the evaluation run began. That threshold controls the re-plan rate, which is itself one of the headline agent-specific measures, so sweeping it on the evaluation set would be tuning on the test set and would invalidate the very metric it was tuned to improve. In the event the sweep was run over this slice only after the evaluation, on confidences produced at the configured value, and Chapter 5 reports what such a post-hoc sweep can and cannot establish; the slice’s disjointness is what keeps even that diagnostic reading free of the evaluation questions. Table 2.1 summarises the two corpora as built.

Table 2.1: Corpus statistics after preprocessing, read from `{dataset}_corpus_stats.json`. Passages are de-duplicated by content under a canonical title-derived id; sentence splits are the datasets’ own and are persisted verbatim, because gold supporting facts index into them by position and re-splitting anywhere downstream would renumber the ground truth while every metric still looked plausible. - marks a statistic the build did not record.

Statistic	hotpotqa	twowiki
Questions in split	7 405	12 576
Paragraphs read	73 700	125 760
Passages after de-duplication	66 581	54 957
Identical paragraphs dropped	7 119	68 186
Title collisions resolved	0	2 617
Sentences	269 602	202 759
Mean sentences per passage	4.05	3.69
Mean passage length (chars)	531.9	427.2
Mean sentence length (chars)	130.6	115.1
Questions with gold triples	0	12 576
Gold supporting facts	18 005	30 687
Supporting facts unresolvable	1	0
Evaluation sample	250	250

Table 2.2: Composition of the frozen evaluation slice against the full split. The slice is stratified on question type and sampled once, with seed 42, by `scripts/sample_eval_set.py`; its SHA-256 is recorded in every run’s `meta.json` and it is never re-sampled at run time. HotpotQA’s validation split is entirely `level=hard`, so difficulty gives one stratum and a silently unstratified sample; question type is the informative split there.

Question type	Split <i>n</i>	Split share	Sample <i>n</i>	Sample share
<i>hotpotqa</i>				
bridge	5 918	0.799	200	0.800
comparison	1 487	0.201	50	0.200
<i>total</i>	7 405	1.000	250	1.000
<i>twowiki</i>				
bridge_comparison	2 751	0.219	55	0.220
comparison	3 040	0.242	60	0.240
compositional	5 236	0.416	104	0.416
inference	1 549	0.123	31	0.124
<i>total</i>	12 576	1.000	250	1.000

Chapter 3

Retrieval Methodologies

3.1 Classical Foundations

The agentic components described later do not replace classical retrieval; they schedule it. Every sub-query the Planner emits is ultimately answered by one of the three mechanisms below, so their individual blind spots determine what the agent layer has to compensate for.

3.1.1 Sparse Retrieval: BM25

BM25 [6] scores a document d against a query q by summing, over the query terms, a saturating function of term frequency weighted by inverse document frequency:

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}.$$

The saturation parameter k_1 controls how quickly repeated occurrences stop adding score; the length normalisation parameter b controls how strongly long documents are penalised. The values used here, $k_1 = 0.9$ and $b = 0.4$ (`retrieval.sparse`), sit below the classical 1.2 and 0.75 deliberately: the indexed unit is a single Wikipedia lead paragraph averaging 4.05 sentences and 531.9 characters, not a full document. At that length repetition carries little extra evidence and length differences are largely incidental, so heavy normalisation would penalise informative paragraphs for being informative.

BM25 matches rare surface forms exactly, needs no training, is interpretable term by term, and answers in milliseconds. It also supplies the analysed lexicon and per-term document frequencies that the routing rules of Section 3.3.2 consume, which nothing else in the system can provide. Its weakness is vocabulary mismatch: a query and a relevant passage sharing no surface term after stemming score zero against one another, however obviously they concern the same thing.

3.1.2 Dense Retrieval

A dense retriever embeds queries and passages independently into a shared vector space and ranks by similarity. This work uses `bge-small-en-v1.5` [11], a 384-dimensional bi-encoder, with vectors L2-normalised so that inner product equals cosine similarity, and an exact FAISS flat inner-product index. Exact rather than approximate search is affordable because the corpora are small — 66,581 and 54,957 passages — and it matters because any recall the system fails to achieve is then attributable to the retrieval method rather than to the data structure. The model is used asymmetrically, as its authors intend: passages are encoded bare, queries are prefixed with `"Represent this sentence for searching relevant passages: "`.

Its strength is paraphrase, exactly the case BM25 cannot see; its weakness is the mirror image. Query and passage never meet, and each is compressed into 384 dimensions before comparison, a bottleneck that preserves topic far better than identity — so two similarly named people or two films sharing a title are liable to be conflated. The two failure modes are close to complementary, which is the standing argument for combining them.

3.1.3 Hybrid Fusion and Reranking

The two ranked lists are combined by Reciprocal Rank Fusion (RRF) [1], which discards the scores and combines positions:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60,$$

with k from `retrieval.hybrid.rrf_k`. Rank-based fusion is preferred over score normalisation because BM25 scores are unbounded and depend on corpus statistics and query length, while cosine similarities from a normalised encoder occupy a narrow band near the top of $[-1, 1]$: the two share no scale and no principled conversion. Normalisation manufactures one, but the fused score then depends on the shape of the candidate list, so a single outlier at the tail can reorder the head. Ranks are ordinal and scale-free, and the ordinal information is the part on which the two retrievers can meaningfully be said to agree.

Fusion improves recall but leaves head precision largely to chance, which is what reranking addresses. A cross-encoder — `ms-marco-MiniLM-L-6-v2` here — concatenates query and candidate and runs full self-attention over the pair, so their terms interact directly. It is more expressive than a bi-encoder for the same reason it is more expensive: nothing about the passage can be precomputed, so scoring n candidates costs n forward passes and admits no index. That cost fixes the depth parameters, `top_n` = 50 before reranking and `top_k` = 10 after. Reranking is also often decorative, so when the fused top-1 already leads the runner-up by a relative RRF margin of at least `rerank_margin_gate` = 0.15 the fifty forward passes cannot change the head of the ranking and are skipped — a saving counted in the trace rather than assumed.

3.2 Why Single-Shot Retrieval Fails on Multi-Hop Needs

Everything in Section 3.1, up to and including the cross-encoder, computes a scoring function $s(q, d)$. The three methods differ enormously in expressive power — from a bag-of-words sum, through a 384-dimensional dot product, to full cross-attention over the pair — but they do not differ in their arguments. All of them assume that the relevance of a document is determinable from the query and that document. For a large class of information needs this assumption is correct and unremarkable. For a bridge question it is false, and the consequence is not that these systems perform poorly but that they are solving the wrong problem.

A worked example. Consider question `5a7793275542992a6e59df04`, drawn from the HotpotQA evaluation sample of Section 2.6:

Which country is the firm that owns Babycham located?

The gold answer is *Australia*, and the two gold supporting facts are the opening sentences of two passages:

Babycham, sentence 0 — “Babycham is the trade name of a light, sparkling perry invented by Francis Showering, a brewer in Shepton Mallet in Somerset, England; the name is now owned by Accolade Wines.”

Accolade Wines, sentence 0 — “Accolade Wines is a major global wine business with headquarters in South Australia.”

The first hop is easy, and easy in a way that flatters BM25 specifically: “Babycham” is a near-unique term in a 66,581-passage corpus, so lexical matching on that term alone puts the correct passage at rank one. This is precisely the situation routing rule R4 of Section 3.3.2 exists to detect — though, as that section records, in the evaluated runs the Planner’s own tool hint settled every routing before R4 could be consulted.

The second hop is not merely harder. Compare the query terms — *country*, *firm*, *owns*, *Babycham*, *located* — against the second gold sentence. Not one of them occurs in it. BM25 has nothing to match and assigns no score on any discriminative term. A bi-encoder produces a non-zero similarity, but the query embedding is dominated by *Babycham* and a possessive relation, while the passage embedding says *wine business*, *global*, *South Australia*. The passage sits among the corpus’s other company profiles, and nothing in it distinguishes it from any of them with respect to this query.

Why the reranker does not close the gap. There are two reasons, and only the second is fundamental. The first is mechanical: a cross-encoder reorders a pool it is handed, so a passage that never enters the pool cannot be recovered by reordering, and widening the pool postpones the problem rather than solving it.

The second reason survives any pool width and any model capacity. Suppose the *Accolade Wines* passage is placed in the pool by fiat, and suppose the cross-encoder is arbitrarily large and perfectly trained. It is presented with the pair (question, passage) and asked to judge relevance. The information that would justify a high score — that Accolade Wines is the firm that owns Babycham — is present in neither element of that pair. It is asserted in a third document. The relevance of the second-hop passage d_2 therefore does not factorise as a function of the query–document pair; it is a conditional quantity $\text{rel}(q, d_2 \mid d_1)$ depending on a document the ranker was never given. Increasing model capacity changes the function class being searched, not the arguments the function receives, so a better ranker is a better approximation of the wrong quantity. This is why the failure is properly described as a specification error rather than a ranking error, and it is the single fact from which the rest of this chapter follows.

What would work. The query that retrieves *Accolade Wines* trivially is “Where is Accolade Wines located?” — a query with the same rare-key property that made the first hop easy. It cannot be written in advance, because the string “Accolade Wines” appears nowhere in the user’s question; it exists only as the output of the first retrieval. The missing capability is therefore not ranking quality but the ability to issue a second query whose text is conditioned on the result of the first. That is a control structure, not a scoring function, and no single-shot architecture can express it however good its scorer is.

Scope of the argument. This failure mode is not an edge case. Bridge questions are 5,918 of HotpotQA’s 7,405 validation questions, or 79.9%, and 2Wiki’s compositional, inference and bridge-comparison types — 9,536 of 12,576 questions — share the same conditional structure. Comparison questions fail differently, and the distinction is worth stating rather than blurring: there both gold passages are individually retrievable from the original query, since both entities are named in it, so the difficulty is coverage at a fixed cut-off plus the fact that the answer is the output of an *operation* over two passages rather than the content of either. That is a weaker claim, and Section 3.3.1 accordingly gives the two shapes different plan topologies. How far the single-shot baselines fall below the achievable ceiling is a measurement, reported in Chapter 4; the point here is that the ceiling exists by construction and cannot be lifted by tuning.

3.3 Agentic Retrieval

The remedy indicated above is a retrieval process that carries state between rounds. That is the common thread through the agentic IR literature: treating a language model as an autonomous agent that plans, invokes tools and revises [14, 10], whether the domain is scientific literature [3], multimodal collections [7], telecommunications networks [13] or geospatial workflows [4]. The four agents below cover, between them, the five tasks of the assignment brief’s Table 1.

3.3.1 Query Decomposition and Refinement

The Planner converts a question into a directed acyclic graph of at most five sub-queries. Each node carries its own text, an explicit dependency list, an intent and an expected answer type, and may contain template placeholders of the form `{{q1.answer}}`, `{{q1.entity}}` or `{{q1.title}}`. Execution proceeds in level-synchronous topological order, and a node’s placeholders are substituted from the state of its dependencies immediately before it is routed and retrieved. The second-hop query is therefore a genuinely different string at retrieval time from the one the Planner emitted — exactly the capability Section 3.2 identifies as missing — and because the resolved text differs lexically from the template, routing is recomputed after substitution rather than inherited.

This is the same territory as Self-Ask [5], which interleaves follow-up questions inside a single generation, and IRCOT [9], which interleaves retrieval with chain-of-thought steps. Four differences matter. The plan here is an explicit, validated data structure rather than a decoding trajectory: cycles, dangling dependencies, over-long plans and unbound placeholders are repaired mechanically before execution and every repair is recorded, so a model-generated plan is never executed on trust. It is a graph rather than a chain, so a comparison question yields two independent roots retrieved in the same level where a linear follow-up mechanism must serialise them. Query expansion rides inside the same planner response as a per-node list of at most two rewrites, fused with RRF at retrieval time, so expansion costs *zero* additional model calls. And the plan can be revised on evidence supplied by a downstream agent (Section 3.3.4), which no single-pass decoding trajectory can do.

Two Planner decisions followed from measurement. The plan’s **strategy** label is derived from the shape of the validated graph and overwrites whatever the model reported: on eight representative questions at temperature 0 with a constrained schema, **qwen3:8b** produced structurally correct decompositions 8 times out of 8 but labelled the strategy **single_hop** in 7 of those 8, including unambiguous bridge and comparison cases. It builds the graph well and names it badly, and since that label gates a deterministic comparison shortcut, trusting it would silently disable one; both labels are logged, so their disagreement rate is itself a reportable result about the self-knowledge of a small local model. Separately, the model reliably emits a final *combiner* node restating the question and depending on every other node, whose answer comes from composing its dependencies rather than from any passage; detecting and skipping it removes a third of all retrieval on a three-node comparison plan.

The Planner degrades along a deterministic ladder rather than failing: a comparison template, then a two-node bridge template, then an identity plan routed to hybrid search with reranking. It is tempting to describe that floor as the **hybrid_rerank** baseline, and an earlier draft of this report did; the description is wrong, and the ablation grid of Chapter 4 is what shows it to be wrong. An identity plan still passes through synthesis and verification, so the system with the Planner removed is a one-node agentic system that answers, cites, and can still re-plan — not a ranker. What the ladder guarantees is narrower and still worth having: retrieval itself can never be worse than a single hybrid query with reranking, so a retrieval deficit against that baseline is a judgement about what the Planner chose to ask, never an infrastructure failure.

3.3.2 Tool-Augmented Retrieval

Tools are declared in a registry as a name, a callable, an argument schema and a natural-language description. Seven are exposed across two families: `bm25_search`, `dense_search`, `hybrid_search` and `rerank` over the indexes, and `kg_link`, `kg_neighbors` and `kg_path` over the entity graph. The same description strings that document a tool to a human are the strings injected into the tool-selection prompt, so model-facing documentation and executable registry cannot drift apart — a small property that removes an entire class of silent bug in which an agent is told about a tool that no longer behaves as described.

Selection among the retrieval tools is a seven-rule decision table over features of the resolved sub-query: token count after stopword removal, presence of a quoted span, number and token share of capitalised runs, out-of-vocabulary rate against the BM25 lexicon, minimum document frequency among content terms, and whether the node has dependencies. The rules encode Section 3.1 directly. A planner hint is honoured (R1); an out-of-vocabulary rate above 0.34 routes to dense search (R2), because BM25 cannot score terms absent from its lexicon; short quoted or proper-noun-dense queries (R3) and queries containing a term of document frequency three or fewer (R4) route to BM25, a very rare term being a near-unique key — the Babycham case above; dependent nodes (R5) and long or entity-free queries (R6) route to hybrid, since a bridge-filled query mixes one precise entity with paraphrased relation text and neither channel alone suffices; and a default row (R7) makes the table total.

One fact about the table has to be stated before its cost is argued for, because it changes what the argument is about. Rule R1 honours a tool hint the Planner may attach to a sub-query, and the decision-table is evaluated first match wins. In the headline HotpotQA run the Planner attached a hint to every one of the 854 sub-queries it emitted, so R1 fired on all 575 routing decisions the trace records and rules R2–R7 never executed once. All seven rules are implemented and unit-tested; on this data six of them are dead code. That is not a defect in the rules but a finding about the division of labour between the two agents: the schema-constrained planning call, which the system pays for anyway, already carries a per-node tool choice, and the lexical features the remaining rules consult were never needed to decide anything. The cost argument below therefore describes a one-rule router in practice, and the saving it claims is the saving of not asking the model a second time for a decision it made in its first call. Whether the model’s hints agree with what the lexical rules would have chosen is a measurable question and is recorded as an open one in Chapter 4.

The table costs zero model calls, and that deserves stating as an argument rather than as an optimisation. An agent that spends a language-model call on a decision a rule already settles is spending the user’s latency budget, not demonstrating intelligence. At the measured 52 tokens per second a schema-constrained call costs roughly 3.75s, whereas the routing rule is a handful of look-ups against an already-loaded lexicon; multiplied across sub-queries, 250 questions, two datasets and nine configurations, that is hours of wall clock purchased for a decision the default row would often have made identically. Autonomy is a property of which decisions a system makes without a human, not of which decisions it routes through a transformer.

The claim is nonetheless empirical, and the experiment that would test it was designed but has not been run. The design was to clear the retriever’s `heuristic_shortcut` flag, which routes every sub-query through a tool-selection prompt instead of the table, run that configuration on the disjoint 50-question calibration slice, and report the rule–model agreement rate, the change in `nDCG@10` and the change in wall clock. No such run exists under `results/`, so no agreement rate is reported anywhere in this document. There is also a reason the experiment cannot be run as the code stands: the retriever consults the planner hint before it consults the flag, so a sub-query carrying a hint is routed by R1 whether or not the shortcut is enabled, and with a hint on every sub-query the selection prompt is unreachable. Making the model router reachable requires suppressing the Planner’s hints as well, which is a different experiment from the one designed and one that would test the lexical rules R2–R7 rather than the router the system

actually used. The paragraph above is therefore a design argument, not a measured result, and Chapter 4 says so again where the cost figures are reported.

This is also where reinforcement learning would enter, and why it does not enter here. Tool routing is a policy over a small discrete action space with an observable reward, which is textbook territory for a contextual bandit or a policy-gradient learner. The obstacle is specific and measurable: the reward would have to derive from gold supporting facts, and the only data carrying those labels is the evaluation data, so an on-policy learner would be training on the test set and any gain would be uninterpretable. The one leakage-free signal available is the 50-question calibration slice, far too small to fit a policy that reliably beats a seven-rule table. Reinforcement learning is therefore listed as future work in Chapter 5 rather than claimed here.

3.3.3 Knowledge Graph Traversal

The KG Navigator operates on the entity graph built in Section 2.5. Entity linking is an alias-trie longest match over normalised titles and costs no model call. Traversal is bounded at two hops and twenty-five neighbours per node (`agents.kg.max_hops`, `max_neighbors`) and expanded in deterministic order of descending edge weight, so a repeated run explores the graph in exactly the same sequence.

The highest-value operation is bidirectional breadth-first search: given two or more linked seeds it expands from each and reports the meeting node as the *bridge entity*. Applied to the example of Section 3.2, the alias “Accolade Wines” occurring in the text of the *Babycham* passage is an edge in the graph by construction, so the bridge entity that the retrieval stack could not surface by ranking is recovered by a graph walk in milliseconds, and recovered without a generation step where an equivalent model call would cost seconds. This is the concrete sense in which graph traversal is not decorative: it is a second, structurally different mechanism for the one operation single-shot retrieval provably cannot perform.

Traversal is also the most explainable component, because the path *is* the explanation. Every edge stores the document identifier, the sentence identifier and the sentence asserting it, so a traversal result enters the evidence pool at the same sentence granularity as a retrieved passage, is cited in the same way, and is scored by the same supporting-fact metrics; a graph result that could not be cited would be unfalsifiable. The limits should be equally clear: the graph is induced from mention co-occurrence, not curated from an ontology, and 23.2% of HotpotQA edges and 21.0% of 2Wiki edges carry no label beyond `mentions`, so relation semantics are weak and the structure is best understood as a navigation aid rather than a knowledge base. Comparison questions are handled accordingly: with two seeds and no path expected, each seed’s attribute-bearing sentences are returned instead of forcing a spurious connection, and an empty result is a legal outcome.

3.3.4 Self-Reflective Validation

The Verifier receives the Synthesizer’s answer with the evidence pool and decides whether it is supported. Cited identifiers that do not exist are recorded as hallucinated citations and contribute no support, which alone catches a common failure of small local models. A natural language inference model, `cross-encoder/nli-deberta-v3-base`, is then run with each cited sentence as premise and the answer sentence as hypothesis, and the maximum probability of entailment becomes the NLI support; the Synthesizer emits that answer sentence as a separate field precisely so the hypothesis is a well-formed declarative sentence, which is what an NLI model is trained on, rather than the usual concatenation of question and answer. Two further signals are computed — citation grounding, the fraction of claims with at least one valid supporting citation, and retrieval agreement, the fraction of executed sub-queries whose top-ranked title appears among the cited evidence — and the four combine as

$$\text{conf} = 0.45 \text{ nli} + 0.25 \text{ citation} + 0.15 \text{ retrieval} + 0.15 \text{ llm},$$

with weights from `agents.verifier.weights`, renormalised when the model term is absent.

That last term is gated: the adjudication call is issued only when the confidence computed without it falls inside $[0.40, 0.70]$, the threshold of 0.55 plus or minus an uncertainty band of 0.15, because confident acceptances and confident rejections cannot have their verdict changed by it. Above the threshold the verdict is **accept**. Below it, with re-plans remaining, the verdict is **revise** and the Verifier emits a re-plan directive carrying what could not be established, which sub-queries returned nothing useful, which entities are already covered, and — the field that makes the loop work at all — the normalised texts of every sub-query already tried. Without that ban list, an 8B model told only that its previous attempt was insufficient reliably re-emits the same decomposition and burns both re-plans for nothing. Below the threshold with no re-plans left the verdict is **abstain**, a label rather than a refusal: the best candidate is still returned, so answer metrics remain computable.

A verifier can be wrong in two directions and the design has to survive both. Too strict, it triggers re-planning that costs latency and risks replacing a good answer with a worse one; too lenient, it rubber-stamps errors while reporting high confidence. Answer selection at termination is therefore the argument maximising confidence over *all* cycles, ties broken toward the earlier and cheaper answer, so a bad re-plan cannot silently overwrite a good first attempt; and the error taxonomy of Chapter 4 contains explicit `verifier_false_accept` and `verifier_false_reject` categories, computed post hoc against gold supporting facts, so the loop is falsifiable rather than merely asserted. The threshold of 0.55 was set a priori. The design called for it to be swept on the disjoint calibration slice of Section 2.6 and frozen before the evaluation run, because it controls the re-plan rate and tuning it on the evaluation set would corrupt one of the headline agent metrics; what was actually done is a post-hoc sweep over that slice, reading confidences that a single run at 0.55 had already produced, and Chapter 5 reports both what that sweep found and why it is not the experiment the design specified.

3.4 The Role of the Language Model

The language model is the component that generalises, and everything else in this system exists to keep the number of calls small and their inputs short. Three decisions genuinely require it: mapping an unseen compound question onto a dependency structure, which is open-ended and cannot be enumerated by rules; composing a short answer span from heterogeneous evidence sentences, which requires reading; and adjudicating the narrow band where the numeric confidence signals disagree, which requires judgement arithmetic does not supply. Everything else is deterministic — tool routing, entity linking, graph traversal, entailment, evidence deduplication and ranking, comparison arithmetic over parsed dates and numbers, and query expansion, which rides inside the planner call. Each is a decision over a small closed action space with a cheap and reliable feature, where a model call would add latency and a new failure mode, malformed output, without adding accuracy. The worst case over a whole question is therefore 15 calls against a cap of 20, with 4–7 typical, and two are reserved so synthesis and verification always run even when the Planner has exhausted the budget.

Serving that model locally shaped the design more than any other single factor, and Chapter 4 gives the full configuration. Two consequences belong here because they changed the method rather than the deployment. Throughput of 52 tokens per second makes prompt length a latency cost, which is why the re-plan prompt passes retrieved *titles* rather than retrieved text, holding it to a few hundred tokens instead of several thousand. And **qwen3** is a hybrid reasoning model that emits a `<think>` block by default, whose measured effect is worse than added latency: with thinking enabled and a 128-token prediction budget, the model spent the *entire* budget on a 503-character reasoning block and produced no answer at all. Thinking does not merely delay the response, it starves it, so suppression is applied three ways and whatever leaks through is logged. Structured output is parsed down a five-rung ladder ending in a repair prompt, and

parse failures are counted per agent — because if the Planner’s rate is high, what the results measure is partly the fallback ladder rather than the model, and the report has to say so.

3.5 Architectural Summary

Figure 3.1 shows the resulting system. Read left to right it is a pipeline: a Planner decomposes, a Retrieval Agent routes each sub-query to the indexes, a KG Navigator recovers bridge entities from the entity graph, a Synthesizer composes a cited answer, and a Verifier scores it. The edge that makes it an agent rather than a pipeline is the dashed one drawn above: below a confidence of 0.55 the Verifier returns a re-plan directive to the Planner instead of returning an answer, and the system tries again with an explicit record of what it already knows and what it must not repeat. Every other component in this chapter is standard IR machinery arranged carefully; that backward edge is the contribution, and Chapter 4 isolates its effect by ablation rather than assuming it.

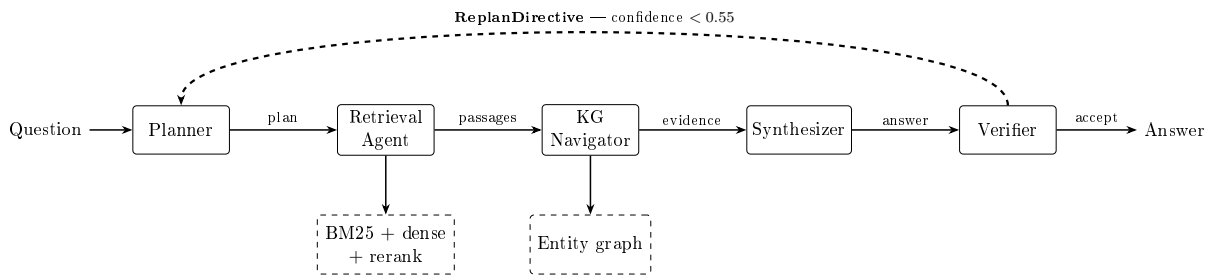


Figure 3.1: The four-agent architecture. Solid arrows are the forward retrieval path; dashed boxes are offline artefacts built in Section 2.5. The dashed backward edge is the Verifier-to-Planner feedback loop — the only information that flows against the pipeline, and what distinguishes this system from a retrieval-augmented generation pipeline.

Mapped onto the assignment brief’s Table 1, the Planner covers autonomous query refinement and multi-step retrieval planning, the tool registry and routing table cover tool-augmented retrieval, the KG Navigator covers knowledge graph traversal, and the Verifier covers self-reflective validation and fact checking; Table 1.1 in Chapter 1 gives the mapping row by row with the sections that specify and measure each. Five tasks are addressed by four agents because refinement and planning are the same act performed at different points in the loop: once before evidence exists, and again after the Verifier has said what is missing.

Chapter 4

System Implementation and Evaluation

Chapter 3 argued that a multi-hop information need cannot be served by a single retrieval round, and set out the four agents that answer it. This chapter describes how those agents are built, how the system is measured, and what the measurements show. It closes with an error analysis, because a system that is right for the wrong reasons is not a result.

4.1 System Architecture

The brief names four modules — a query planner, a tool-invocation manager, a retrieval engine and a validation component. They map onto the four agents of Chapter 3 and an orchestrator that sequences them.

The orchestrator is an *explicit* state machine rather than a chain of function calls. Nine states (`INIT`, `PLAN`, `EXECUTE`, `AGGREGATE`, `SYNTHESIZE`, `VERIFY`, `REPLAN_GATE`, `FINALIZE`, `DONE`) and fourteen labelled transitions are enumerated in a table in the source, and the transition sequence taken by each question is written into its trace. This costs more code than implicit control flow and buys two things. First, the mechanism is inspectable: the claim that the system re-plans is checkable against a recorded path rather than asserted. Second, every exit is named, so a question that terminates early is attributable to a specific guard rather than disappearing into a generic failure count.

Only one transition runs backwards. `REPLAN_GATE` returns to `PLAN` when the Verifier’s confidence falls below threshold, carrying a `ReplanDirective`. Removing that single edge reduces the system to a retrieve-then-generate pipeline, which is why the ablation in §4.7 isolates it directly.

4.1.1 Guarding against non-termination

An agent that can send work backwards can loop. Five guards are evaluated in order at `REPLAN_GATE`, and the first to fail names the termination reason recorded in the trace: the re-plan count must be below `max_replans` (2); the iteration count below `max_iterations` (6); at least three LLM calls must remain; elapsed time must be under 60% of the 300 s wall-clock budget; and the directive must be informative, meaning it names at least one missing item, failed sub-query or suggestion.

A sixth check applies to the plan that comes back. A local 8B model told that its previous attempt was insufficient will frequently re-emit the same decomposition in different words. The system therefore normalises each sub-query — lowercased, punctuation and placeholders stripped, stopwords removed — and rejects a revision whose sub-queries pair with the previous plan’s at a Jaccard similarity of 0.85 or above. Without this, both permitted re-plans can be consumed re-running an identical plan, and the feedback loop becomes pure cost.

4.2 Implementation Details

4.2.1 Indexing

Both corpora are indexed twice. The sparse channel is BM25 over stemmed terms ($k_1 = 0.9$, $b = 0.4$); the dense channel is `bge-small-en-v1.5` [11] embeddings in a FAISS inner-product index over L2-normalised vectors, which makes inner product equivalent to cosine similarity. Both are exact: at this corpus size an approximate index would trade recall for a speed advantage the system does not need.

For 2WikiMultihopQA’s 54,957 passages the sparse index builds in 4.1 s and the dense index in 97.6 s (95.8 s of which is GPU encoding), occupying 103.6 MB on disk against HotpotQA’s 129.1 MB. The asymmetry between the two build times is the reason the encoder is given the GPU here and taken off it later (§4.2.2).

The knowledge graph is induced from passage text alone. For HotpotQA, 417,881 candidate mentions yield 123,759 retained edges over 64,733 nodes in 49.4 s, with 446 hub nodes pruned; for 2WikiMultihopQA, 161,572 candidates give 63,073 edges over 52,959 nodes in 32.9 s with 172 hubs pruned. 76.8% and 79.0% of edges respectively carry a verb-anchored relation label; the remainder are recorded as untyped co-mention, which the traversal still uses for connectivity.

That the graph is built from passage text and never from 2WikiMultihopQA’s gold evidence triples is not a stylistic preference. Those triples are the answer key for the very capability the KG Navigator is being measured on, and a graph built from them would make every KG-attributed gain an artefact. The constraint is enforced by a test that fails if the graph builder opens any file whose name contains `evidence` or `triple`.

4.2.2 Local inference

All generation runs on `qwen3:8b` at Q4_K_M quantisation through Ollama, on an RTX 5060 laptop GPU with 8 GB of VRAM. No component of this project contacts a paid API, and a test enforces that no module outside the model client imports an HTTP library at all.

Measured throughput is 52 tokens per second with the model warm, and a schema-constrained planning call averages 3.75 s. Two properties of the hardware shape the design more than the throughput figure does.

The first is that the card is effectively full. The model and its 8192-token KV cache occupy 6,707 of 8,151 MiB with the desktop’s own allocation included, leaving roughly 1.4 GiB. The three sentence-transformer encoders — query embedding, cross-encoder reranking and NLI — therefore run on CPU at query time, where they cost well under a second per question against a 300 s budget. The GPU is used for encoders only during offline index construction, with the language model unloaded. An encoder co-resident with the model produces an out-of-memory failure partway through an evaluation run, which is the most expensive failure mode available: hours of work lost with partial results.

The second is that the first call after a model load costs approximately 9.5 s in CUDA warm-up, evaluating a 19-token prompt. Warm, the same work takes under two seconds. Latency figures in this chapter are therefore reported from runs with the model resident. One thing the harness does not do should be stated because the configuration file suggests otherwise. `config.yaml` declares a response cache under `llm.cache`, and the trace schema carries a cache-hit counter; no cache is implemented. Nothing reads or writes the declared file, every call in every run reported here went to the model, and the counter is constitutively zero. Every latency in this chapter is consequently an uncached measurement, which is the property a timing figure needs, but it holds because the feature is absent rather than because it was disabled.

4.2.3 Structured output and its failure mode

Every agent that calls the model does so with a JSON schema and parses the result through a five-rung ladder: direct parse; strip reasoning blocks and code fences; a string-aware balanced-brace scan; trailing-comma repair; and finally a re-prompt carrying the schema and the malformed output. On a measurement over eight representative multi-hop questions, all eight parsed on the first rung with no retries and no reasoning leakage.

Suppressing that reasoning is not cosmetic. **qwen3** is a hybrid reasoning model, and with thinking enabled and a 128-token generation cap it spent the entire budget on a 503-character reasoning block and produced no answer at all. Thinking does not merely add latency here; it starves the response. It is disabled three ways: an API flag, a system-prompt directive, and stripping on the response, with the size of any leaked block recorded so the report can state how often the suppression fails rather than assume it.

4.2.4 Budget enforcement

Each question carries a budget of 20 LLM calls, 6 iterations and 300 s. Budget is checked and never raised as an exception: a request for a call returns a boolean, and an agent that is refused takes its deterministic fallback. Exception-driven budget control would make “ran out of calls” indistinguishable from “crashed” in the trace, and those are different findings.

Two of the twenty calls are reserved. Non-privileged callers — the planner, answer extraction and entity linking — see the reserve subtracted from their remaining allowance, while the synthesizer and verifier see the true remainder. Without this, a planner that decomposes aggressively can consume the whole budget and leave nothing with which to answer, which appears in the results as a system failure rather than as the budget policy it actually is.

The worst case is 15 calls against the cap of 20, and typical usage is four to seven. The headroom is deliberate: the cap is a safety net, not a tuning knob. Most of the saving comes from work the language model never does. Tool routing is a seven-rule decision table costing nothing — in practice a one-rule table, since the Planner’s hint decided every routing in the evaluated runs (§3.3.2); graph traversal is bidirectional breadth-first search; answer extraction reaches the model only at the fourth of five rungs, and only for sub-queries that some later sub-query depends on; and the verifier’s adjudication call is issued only inside a narrow band of uncertainty around its threshold.

4.3 Experimental Setup

All results use the frozen 250-question stratified samples described in §2.6, drawn once with seed 42 and materialised to disk with their SHA-256 recorded, so that re-sampling cannot silently occur between runs. A disjoint 50-question calibration slice, sharing no question identifier with the evaluation set, is reserved for tuning the verifier threshold.

Every run writes three artefacts: a JSON Lines trace with one complete record per question, a **meta.json** capturing the full configuration, library versions, GPU identity, model digest and seed, and a flat **metrics.csv**. The harness checkpoints by appending and skipping question identifiers already present, so a run interrupted at question 194 resumes at 194. Given that a full agentic configuration takes hours on this hardware, that is a correctness property rather than a convenience.

Sorting is deterministic throughout, with an explicit tiebreaker — ranking by $(-\text{score}, \text{doc_id})$ rather than by score alone — because two passages with identical scores would otherwise be ordered by dictionary insertion and change the reported metrics between runs.

The implementation is covered by 248 automated tests. Three enforce properties that would otherwise fail silently: that no module outside the model client can reach the network, that no

agent reads the gold answer carried on the question state for tracing, and that the knowledge graph builder never opens a gold evidence file.

4.4 Baselines

Five baselines run from weakest to strongest.

bm25_only and **dense_only** isolate the two retrieval channels. **hybrid_rerank** fuses them with reciprocal rank fusion [1] and reranks the top 50 with a cross-encoder; it is the strongest non-agentic system available here and the one the agentic system must actually beat. It is deliberately not weakened: the margin gate that lets the agentic retriever skip reranking when the fused top result is already clearly ahead is switched off in the baseline, so every question receives all 50 cross-encoder passes. That gate is an efficiency feature of the agentic system, and allowing it to skip work inside the opponent would hand the comparison a head start.

naive_rag retrieves once with the same hybrid-and-rerank pipeline and then makes a single generation call. Because it inherits its retrieval from **hybrid_rerank** rather than reimplementing it, the two agree exactly on every retrieval metric, and the difference between them is one generation step and nothing else.

self_ask is the published multi-hop method [5]: the model is asked whether a follow-up question is needed, retrieval is issued for it, and the loop repeats before a final answer. It has decomposition but no verification and no re-planning, which makes the gap between it and the full agentic system the closest available estimate of what the backward edge is worth.

4.5 Evaluation Metrics

Three families are reported together.

Retrieval quality is Recall@{2, 5, 10}, nDCG@10 and MRR, computed with **pytrec_eval**. That library wraps the same evaluation core as **trec_eval**, which produced the published BEIR figures [8] this work compares against; a reimplementation would risk breaking that comparability for no gain. Questions without relevance judgments are excluded rather than scored as zero.

Answer quality is exact match and token-level F1 under the official HotpotQA normalisation [12], together with supporting-fact EM and F1 as set metrics over (*title*, *sentence id*) pairs, and the joint measures. The official yes/no short-circuit is retained: without it, token-overlap F1 awards partial credit for answering “no” where the gold answer is “yes”, which is not partial correctness but a wrong categorical answer.

Agent-specific cost is LLM calls, calls avoided by deterministic rules, tool calls, latency, plan depth, re-plan rate and citation grounding. Reporting these beside quality is what the brief asks for, and it is also the only honest way to present an agentic system: a method that gains three points of F1 for twenty times the compute has not obviously won, and a table that omits the denominator invites the reader to assume it did. Two definitions are fixed explicitly because they are easy to conflate: re-plan rate is the *fraction of questions* that triggered at least one re-plan, reported alongside the mean count, and citation grounding is averaged only over questions that produced an answer, so that “cited badly” is not merged with “did not answer”.

Differences between systems are assessed with a paired bootstrap over 1,000 resamples. Both systems are scored on the same resampled question set at each iteration; resampling them independently would inflate the variance of the difference and hide real effects. With 250 questions a difference of a few points is frequently indistinguishable from noise, and the intervals are reported so that the reader can see when that is the case.

Table 4.1: Answer quality on `hotpotqa`. Exact match and token-level F1 follow the official HotpotQA evaluation script, including its yes/no short circuit; SP-EM, SP-P, SP-R and SP-F1 are set metrics over $(title, sent_id)$ supporting-fact pairs. n is the number of questions scored. Reference system for the EM and F1 columns: `self_ask`, marked †. **Bold** = significantly better than the reference, $\downarrow$ = significantly worse (paired bootstrap, 95% CI of the paired difference excluding zero). Unmarked differences are not significant. Reference system for the supporting-fact columns: `hybrid_rerank`, marked †. **SP-EM, SP-P, SP-R and SP-F1 are not one protocol**: `agentic_full`, `agentic_no_planner`, `agentic_no_kg` and `agentic_no_verifier` are scored on the sentences they actually cited (mean 1.5 sentences per question), whereas `bm25_only`, `dense_only`, `hybrid_rerank`, `naive_rag` and `self_ask` are scored on their whole evidence pool (mean 6.9) because they emit no citation that resolves to that evidence, so a single SP-F1 is not comparable across the two groups and no significance mark is placed between rows of different protocol; *Pool SP-R* is the recall of the whole evidence pool for every row alike and is the one supporting-fact column compared like for like. - across a whole row marks a configuration that has not been run. `bm25_only`, `dense_only` and `hybrid_rerank` are retrieval-only: they emit a ranking and no answer, so the EM, F1 and Δ F1 columns show - because there is no answer to score, not because the run is absent. Scoring the unattempted answer would report 0.000 and assert a failed attempt that never happened. ‡ **marks a preliminary row**: `agentic_no_kg` ($n = 184$) had not covered the full 250-question evaluation slice when this table was generated, so the point estimates are over the questions completed so far and every delta, p -value and significance mark against it is withheld – a paired test between a prefix and the full slice is not a paired test. These numbers will move; they are not final results. * **marks a score measured differently, not a worse system**: every record of `naive_rag` and `self_ask` cites passage identifiers (p1, p2) that name none of the evidence identifiers the same record stores, because the adapter keys evidence `e1..en`. The citation signal is therefore unusable and `predicted_supporting_facts` falls back to scoring the full evidence pool rather than the cited subset. That fallback favours recall and pays for it in precision, so these scores are not strictly comparable to a system whose citations resolve and which is judged on what it actually cited. Before the fallback was made conditional on citations resolving, the same mismatch produced exact zeros that read as a categorical finding; see `docs/report-audit.md`.

System	n	EM	F1	F1 95% CI	SP-EM	SP-P	SP-R	SP-F1	Pool SP-R	Δ F1 vs. ref. [95% CI]
<i>Baselines (weakest first)</i>										
<code>bm25_only</code>	250	-	-	-	0.012	0.245 $\downarrow$	0.588 $\downarrow$	0.330 $\downarrow$	0.588 $\downarrow$	-
<code>dense_only</code>	250	-	-	-	0.020	0.283 $\downarrow$	0.650 $\downarrow$	0.377 $\downarrow$	0.650 $\downarrow$	-
<code>hybrid_rerank</code>	250	-	-	-	0.016	0.308 $\dagger$	0.706 $\dagger$	0.412 $\dagger$	0.706 $\dagger$	-
<code>naive_rag</code>	250	0.496	0.581	[0.522, 0.641]	0.016*	0.308*	0.706*	0.412*	0.706	-0.022 [-0.060, +0.018]
<code>self_ask</code>	250	0.504 $\dagger$	0.603 $\dagger$	[0.547, 0.659]	0.016*	0.302*	0.696*	0.405*	0.696	-
<i>Agentic system and ablations</i>										
<code>agentic_full</code>	250	0.432 $\downarrow$	0.510 $\downarrow$	[0.451, 0.571]	0.088	0.654	0.406	0.481	0.793	-0.093 [-0.145, -0.039]
<code>agentic_no_planner</code>	250	0.456	0.543 $\downarrow$	[0.489, 0.602]	0.136	0.685	0.431	0.508	0.817	-0.060 [-0.106, -0.009]
<code>agentic_no_kg</code>	184 $\ddagger$	0.418	0.525	[0.456, 0.594]	0.103	0.658	0.407	0.482	0.804	-
<code>agentic_no_verifier</code>	250	0.396 $\downarrow$	0.474 $\downarrow$	[0.418, 0.536]	0.108	0.672	0.414	0.491	0.795	-0.130 [-0.185, -0.074]

Table 4.2: Retrieval quality on `hotpotqa`, computed with `pytrec_eval` over the reciprocal-rank fusion of every sub-query’s ranking and judged against the gold supporting-fact documents. A multi-hop system issues several queries, so “the ranking” is not directly observed; fusing them is how the orchestrator itself pools evidence, and it needs no score calibration between BM25 and cosine. Reference system for the retrieval columns: `hybrid_rerank`, marked †. **Bold** = significantly better than the reference, ↓ = significantly worse (paired bootstrap, 95% CI of the paired difference excluding zero). Unmarked differences are not significant. - across a whole row marks a configuration that has not been run. ‡ marks a preliminary row: `agentic_no_kg` ($n = 184$) had not covered the full 250-question evaluation slice when this table was generated, so the point estimates are over the questions completed so far and every delta, p -value and significance mark against it is withheld – a paired test between a prefix and the full slice is not a paired test. These numbers will move; they are not final results.

System	R@2	R@5	R@10	nDCG@10	nDCG@10 95% CI	MRR
<i>Baselines (weakest first)</i>						
<code>bm25_only</code>	0.576	0.700	0.774↓	0.728↓	[0.696, 0.754]	0.873
<code>dense_only</code>	0.652	0.762	0.816↓	0.787↓	[0.756, 0.817]	0.917
<code>hybrid_rerank</code>	0.696	0.824	0.876†	0.835†	[0.810, 0.859]	0.935
<code>naive_rag</code>	0.696	0.824	0.876	0.835	[0.810, 0.859]	0.935
<code>self_ask</code>	0.688	0.838	0.888	0.837	[0.812, 0.860]	0.929
<i>Agentic system and ablations</i>						
<code>agentic_full</code>	0.602	0.742	0.812↓	0.763↓	[0.734, 0.791]	0.899
<code>agentic_no_planner</code>	0.686	0.820	0.872	0.831↓	[0.804, 0.855]	0.933
<code>agentic_no_kg</code> ‡	0.606	0.764	0.837	0.775	[0.742, 0.806]	0.894
<code>agentic_no_verifier</code>	0.612	0.756	0.796↓	0.758↓	[0.729, 0.786]	0.888

4.6 Results

Tables 4.1 and 4.3 report answer and supporting-fact quality, Tables 4.2 and 4.4 retrieval quality, and Tables 4.5 and 4.6 the agent-specific cost columns. Every figure in these tables is generated directly from the run artefacts by `tables.py` and inserted verbatim; no number in this report is typed by hand from a run, so the tables and the recorded runs cannot drift apart. A configuration that has not been run renders as - rather than as zero, because a zero would read as a measured failure rather than an absent experiment; a retrieval-only configuration renders - in the answer columns for the same reason, since it attempts no answer and scoring the unattempted answer as 0.000 would assert a failure that never happened.

The retrieval ladder behaves as Chapter 3 predicts. On HotpotQA the sparse channel reaches Recall@10 of 0.774 and the dense channel 0.816, while fusing them and reranking reaches 0.876 — an improvement over the better single channel of six points, obtained without any language model involvement. Supporting-fact F1 rises correspondingly from 0.330 to 0.412. The same ordering holds on 2WikiMultihopQA at uniformly lower absolute values, which is consistent with its longer comparison chains and its larger proportion of questions whose evidence is distributed across four passages rather than two.

Answer quality does not follow that ladder, and the first thing to record about it is that the agentic system does not win. On HotpotQA `self_ask` reaches an exact match of 0.504 and a token F1 of 0.603, `naive_rag` 0.496 and 0.581, and `agentic_full` 0.432 and 0.510. On 2WikiMultihopQA the leader is unchanged — `self_ask` at 0.264 and 0.310 against `agentic_full` at 0.224 and 0.267 — though there the full system sits above `naive_rag`’s 0.152 and 0.200 on the point estimates, which is what Chapter 1 predicts of a single retrieve-then-generate step on chains of three and four hops; no paired interval between those two systems is computed, and their deltas against the common reference overlap, so that ordering is not certified. The

Table 4.3: Answer quality on `twowiki`. Exact match and token-level F1 follow the official HotpotQA evaluation script, including its yes/no short circuit; SP-EM, SP-P, SP-R and SP-F1 are set metrics over $(title, sent_id)$ supporting-fact pairs. n is the number of questions scored. Reference system for the EM and F1 columns: `self_ask`, marked †. **Bold** = significantly better than the reference, † = significantly worse (paired bootstrap, 95% CI of the paired difference excluding zero). Unmarked differences are not significant. Reference system for the supporting-fact columns: `hybrid_rerank`, marked †. **SP-EM, SP-P, SP-R and SP-F1 are not one protocol: agentic_full** is scored on the sentences it actually cited (mean 1.6 sentences per question), whereas `bm25_only`, `dense_only`, `hybrid_rerank`, `naive_rag` and `self_ask` are scored on their whole evidence pool (mean 8.9) because they emit no citation that resolves to that evidence, so a single SP-F1 is not comparable across the two groups and no significance mark is placed between rows of different protocol; *Pool SP-R* is the recall of the whole evidence pool for every row alike and is the one supporting-fact column compared like for like. - across a whole row marks a configuration that has not been run. `bm25_only`, `dense_only` and `hybrid_rerank` are retrieval-only: they emit a ranking and no answer, so the EM, F1 and $\Delta F1$ columns show - because there is no answer to score, not because the run is absent. Scoring the unattempted answer would report 0.000 and assert a failed attempt that never happened. * **marks a score measured differently, not a worse system**: every record of `naive_rag` and `self_ask` cites passage identifiers (p1, p2) that name none of the evidence identifiers the same record stores, because the adapter keys evidence `e1..en`. The citation signal is therefore unusable and `predicted_supporting_facts` falls back to scoring the full evidence pool rather than the cited subset. That fallback favours recall and pays for it in precision, so these scores are not strictly comparable to a system whose citations resolve and which is judged on what it actually cited. Before the fallback was made conditional on citations resolving, the same mismatch produced exact zeros that read as a categorical finding; see `docs/report-audit.md`.

System	n	EM	F1	F1 95% CI	SP-EM	SP-P	SP-R	SP-F1	Pool SP-R	$\Delta F1$ vs. ref. [95% CI]
<i>Baselines (weakest first)</i>										
<code>bm25_only</code>	250	-	-	-	0.008	0.208 [↓]	0.551 [↓]	0.273 [↓]	0.551 [↓]	-
<code>dense_only</code>	250	-	-	-	0.020	0.244	0.605	0.314	0.605	-
<code>hybrid_rerank</code>	250	-	-	-	0.016	0.255 [†]	0.622 [†]	0.327 [†]	0.622 [†]	-
<code>naive_rag</code>	250	0.152 [↓]	0.200 [↓]	[0.155, 0.246]	0.016*	0.255*	0.622*	0.327*	0.622	-0.110 [-0.160, -0.063]
<code>self_ask</code>	250	0.264 [†]	0.310 [†]	[0.259, 0.364]	0.016*	0.246*	0.627*	0.318*	0.627	-
<i>Agentic system and ablations</i>										
<code>agentic_full</code>	250	0.224	0.267	[0.216, 0.321]	0.124	0.558	0.367	0.421	0.684	-0.043 [-0.100, +0.014]
<code>agentic_no_planner</code>	-	-	-	-	-	-	-	-	-	-
<code>agentic_no_kg</code>	-	-	-	-	-	-	-	-	-	-
<code>agentic_no_verifier</code>	-	-	-	-	-	-	-	-	-	-

Table 4.4: Retrieval quality on **twowiki**, computed with **pytrec_eval** over the reciprocal-rank fusion of every sub-query’s ranking and judged against the gold supporting-fact documents. A multi-hop system issues several queries, so “the ranking” is not directly observed; fusing them is how the orchestrator itself pools evidence, and it needs no score calibration between BM25 and cosine. Reference system for the retrieval columns: **hybrid_rerank**, marked †. **Bold** = significantly better than the reference, ↓ = significantly worse (paired bootstrap, 95% CI of the paired difference excluding zero). Unmarked differences are not significant. - across a whole row marks a configuration that has not been run.

System	R@2	R@5	R@10	nDCG@10	nDCG@10 95% CI	MRR
<i>Baselines (weakest first)</i>						
bm25_only	0.551	0.641	0.693↓	0.698↓	[0.671, 0.724]	0.910
dense_only	0.605	0.687	0.714	0.751	[0.729, 0.774]	0.981
hybrid_rerank	0.622	0.696	0.727†	0.756†	[0.730, 0.780]	0.967
naive_rag	0.622	0.696	0.727	0.756	[0.730, 0.780]	0.967
self_ask	0.627	0.756	0.797	0.790	[0.765, 0.817]	0.934
<i>Agentic system and ablations</i>						
agentic_full	0.559	0.683	0.727	0.723↓	[0.697, 0.748]	0.917
agentic_no_planner	-	-	-	-	-	-
agentic_no_kg	-	-	-	-	-	-
agentic_no_verifier	-	-	-	-	-	-

direction against **self_ask** is consistent across two datasets and both answer metrics: on the answer string, decomposition with verification loses to decomposition without it.

That sentence can be given a significance level, and the level is not the same on the two datasets. The answer columns of Tables 4.1 and 4.3 are referenced to **self_ask**, the strongest baseline that produces an answer string at all, and the $\Delta F1$ column carries the paired bootstrap of the per-question difference. On HotpotQA **agentic_full** is significantly worse: $\Delta F1$ is -0.093 with a 95% interval of $[-0.145, -0.039]$, which excludes zero, and the exact-match column carries the same marker. On 2WikiMultihopQA the difference is -0.043 with an interval of $[-0.100, +0.014]$, which spans zero, so there the deficit is not established at 95% and should be read as a point estimate pointing the same way. The interval is the paired one, over per-question differences on the identical 250 questions; the per-system F1 intervals printed beside the point estimates overlap on both datasets, but overlap of marginal intervals is the wrong test for paired data and is more conservative than the comparison the table actually makes. An earlier version of these tables referenced the answer columns to **hybrid_rerank**, which attempts no answer, and credited the agentic system with an apparently significant improvement that was its own F1 with a plus sign; that reference is gone, and the retrieval-only rows now show - in the answer columns. One further qualification stands: the two systems are not answering under the same contract, which is the subject of the paragraphs below rather than an excuse for this one.

Retrieval shows the same deficit from the other side (Tables 4.2 and 4.4). **agentic_full** records nDCG@10 of 0.763 [0.734, 0.791] on HotpotQA against 0.835 for **hybrid_rerank** and 0.837 for **self_ask**, and 0.723 [0.697, 0.748] on 2WikiMultihopQA against 0.756 and 0.790; both agentic figures carry the marker for significantly *worse* than the reference. This is less a failure of the retrieval agent than a consequence of what it is asked to rank. The ranking reported for a multi-hop configuration is the reciprocal-rank fusion of every sub-query’s list, and a decomposition deliberately issues narrow queries whose individual lists are each relevant to one hop and largely irrelevant to the other. Fusing them dilutes the top of the combined list, where a single well-formed query reranked by a cross-encoder concentrates it. The 2WikiMultihopQA rows make the mechanism visible: **agentic_full** and **hybrid_rerank** tie exactly on Recall@10 at 0.727 while the agentic nDCG@10 is three points lower. The same documents are being

Table 4.5: Agent-specific cost on **hotpotqa**, per question. *LLM calls* counts logical calls, cache hits included; *Saved* counts calls a deterministic rule replaced, under the definition stated below; *Re-plan rate* is a fraction of questions, not a mean count; *Cite grounding* is averaged only over questions that produced a non-empty answer, so it cannot be inflated by abstentions. **Latency is the median per question, with the maximum beside it**, and every other column is a mean: latency is the one unbounded quantity here, so it is the one a single hung question can dominate, and a mean of it would report that question rather than the system. The maximum is printed rather than trimmed so the outlier stays in view. Latency is only comparable between cold-cache runs. **One or more questions overran the 300 s orchestrator.max_wall_clock_s budget: agentic_full** (65744.3 s on 5a7af32e55429931da12c99c, 18.3 h). The budget is evidently only tested between state transitions, so it cannot pre-empt a call that blocks inside one; this is a system defect and not only a reporting one, and it is why the central column is a median. The arithmetic mean of **latency_s** on such a run is that single question divided by n and describes nothing that happens per question. See docs/report-audit.md. **Saved is recomputed from the step trace under one definition**: a call is saved only when a deterministic rule produced an output that, absent the rule, this configuration would have obtained by an LLM call. The agents’ own counter credits every rule that fired; two of those rules stand in for calls this configuration cannot make – with **heuristic_shortcut: true** the LLM router is unreachable for every sub-query, and with **entity_linker: alias_match** there is no LLM entity linker to skip – and the identity plan of the no-planner ablation is the ablation itself, not a rule gating a planner call. A verifier skip for an empty or already-insufficient answer is a call nobody would make, not one replaced. The extraction ladder, which replaces a real rung-4 LLM call on every rung-1 to rung-3 success, never incremented the counter at all. Per-question decomposition, counted terms in bold, so any other definition can be applied from the same numbers: **agentic_full**: routing 3.31, KG alias 2.54, **verifier gate 1.21**, verifier pointless 0.04, **extraction 1.39** (agents recorded 7.10); **agentic_no_planner**: routing 1.00, KG alias 0.94, **verifier gate 0.64**, verifier pointless 0.01, planner ablation 1.19 (agents recorded 3.78); **agentic_no_kg**: routing 3.24, **verifier gate 1.12**, verifier pointless 0.02, **extraction 1.38** (agents recorded 4.38); **agentic_no_verifier**: routing 2.26, KG alias 1.59, **extraction 1.16** (agents recorded 3.85). *Plan depth* is the depth of the *selected* plan (the cycle FINALIZE chose, **best_cycle**), as docs/architecture.md §6 defines it; the orchestrator’s own metrics block records the *latest* plan’s depth and the two differ wherever a re-plan executed and the first cycle’s answer was kept (**agentic_full** 1.71 selected against 1.56 latest, **agentic_no_kg** 1.73 selected against 1.59 latest). *Re-plan rate* is the fraction of questions on which the verifier triggered at least one re-plan (*trig.*) and the fraction on which a re-planned cycle actually executed (*exec.*); the two differ when the planner’s re-plan was a near-duplicate of a plan already run and transition T2b discarded it, which the trace counts as a re-plan but not as a cycle. Questions with a discarded re-plan: **agentic_full** (4 of 250 questions, executed rate 0.472), **agentic_no_planner** (47 of 250 questions, executed rate 0.000), **agentic_no_kg** (2 of 184 questions, executed rate 0.429). - across a whole row marks a configuration that has not been run. ‡ **marks a preliminary row**: **agentic_no_kg** ($n = 184$) had not covered the full 250-question evaluation slice when this table was generated, so the point estimates are over the questions completed so far and every delta, p -value and significance mark against it is withheld – a paired test between a prefix and the full slice is not a paired test. These numbers will move; they are not final results.

System	LLM calls	Saved	Tool calls	Latency (s) med.	Latency (s) max	Plan depth	Sub-queries	Re-plan rate (trig.)	Re-plan rate (exec.)	Cite grounding
<i>Baselines (weakest first)</i>										
bm25_only	0.00	0.00	1.00	0.0	0.0	1.00	1.00	0.000	0.000	0.000
dense_only	0.00	0.00	1.00	0.1	16.4	1.00	1.00	0.000	0.000	0.000
hybrid_rerank	0.00	0.00	2.00	4.9	36.8	1.00	1.00	0.000	0.000	0.000
naive_rag	1.00	0.00	2.00	8.7	31.5	1.00	1.00	0.000	0.000	0.000
self_ask	1.24	0.00	2.41	4.5	35.7	1.20	1.20	0.000	0.000	0.000
<i>Agentic system and ablations</i>										
agentic_full	4.26	2.60	14.83	28.3	65744.3	1.71	1.80	0.472	0.472	0.960
agentic_no_planner	1.35	0.64	3.94	9.7	37.4	1.00	1.00	0.188	0.000	0.936
agentic_no_kg‡	4.23	2.51	8.03	11.0	42.1	1.73	1.84	0.429	0.429	0.967
agentic_no_verifier	2.06	1.16	10.88	16.5	46.9	1.98	2.34	0.000	0.000	0.000

Table 4.6: Agent-specific cost on **twowiki**, per question. *LLM calls* counts logical calls, cache hits included; *Saved* counts calls a deterministic rule replaced, under the definition stated below; *Re-plan rate* is a fraction of questions, not a mean count; *Cite grounding* is averaged only over questions that produced a non-empty answer, so it cannot be inflated by abstentions. **Latency is the median per question, with the maximum beside it**, and every other column is a mean: latency is the one unbounded quantity here, so it is the one a single hung question can dominate, and a mean of it would report that question rather than the system. The maximum is printed rather than trimmed so the outlier stays in view. Latency is only comparable between cold-cache runs. ***Saved* is recomputed from the step trace under one definition**: a call is saved only when a deterministic rule produced an output that, absent the rule, this configuration would have obtained by an LLM call. The agents’ own counter credits every rule that fired; two of those rules stand in for calls this configuration cannot make – with **heuristic_shortcut: true** the LLM router is unreachable for every sub-query, and with **entity_linker: alias_match** there is no LLM entity linker to skip – and the identity plan of the no-planner ablation is the ablation itself, not a rule gating a planner call. A verifier skip for an empty or already-insufficient answer is a call nobody would make, not one replaced. The extraction ladder, which replaces a real rung-4 LLM call on every rung-1 to rung-3 success, never incremented the counter at all. Per-question decomposition, counted terms in bold, so any other definition can be applied from the same numbers: **agentic_full**: routing 3.68, KG alias 2.32, **verifier gate 1.16**, verifier pointless 0.03, **extraction 1.94** (agents recorded 7.19). *Plan depth* is the depth of the *selected* plan (the cycle FINALIZE chose, **best_cycle**), as **docs/architecture.md §6** defines it; the orchestrator’s own metrics block records the *latest* plan’s depth and the two differ wherever a re-plan executed and the first cycle’s answer was kept (**agentic_full** 1.90 selected against 1.76 latest). *Re-plan rate* is the fraction of questions on which the verifier triggered at least one re-plan (*trig.*) and the fraction on which a re-planned cycle actually executed (*exec.*); the two differ when the planner’s re-plan was a near-duplicate of a plan already run and transition T2b discarded it, which the trace counts as a re-plan but not as a cycle. Questions with a discarded re-plan: **agentic_full** (3 of 250 questions, executed rate 0.392). - across a whole row marks a configuration that has not been run.

System	LLM calls	Saved	Tool calls	Latency (s) med.	Latency (s) max	Plan depth	Sub-queries	Re-plan rate (trig.)	Re-plan rate (exec.)	Cite grounding
<i>Baselines (weakest first)</i>										
bm25_only	0.00	0.00	1.00	0.0	0.0	1.00	1.00	0.000	0.000	0.000
dense_only	0.00	0.00	1.00	0.1	13.9	1.00	1.00	0.000	0.000	0.000
hybrid_rerank	0.00	0.00	2.00	8.3	27.0	1.00	1.00	0.000	0.000	0.000
naive_rag	1.00	0.00	2.00	12.2	29.7	1.00	1.00	0.000	0.000	0.000
self_ask	1.46	0.00	2.86	13.2	40.3	1.43	1.43	0.000	0.000	0.000
<i>Agentic system and ablations</i>										
agentic_full	3.80	3.10	16.00	35.4	121.5	1.90	2.26	0.392	0.392	0.976
agentic_no_planner	-	-	-	-	-	-	-	-	-	-
agentic_no_kg	-	-	-	-	-	-	-	-	-	-
agentic_no_verifier	-	-	-	-	-	-	-	-	-	-

found; they are being found further down, spread across several lists, because each was retrieved in answer to a different question. Decomposition buys evidence coverage and pays for it in fused ranking quality, and a single aggregate ranking metric cannot represent that trade, having been designed for systems that issue one query. One defect in the state contract qualifies the agentic retrieval rows specifically. Sub-query identifiers restart at **q1** in every plan revision, and per-sub-query results are keyed by that identifier, so a re-plan’s **q1** overwrites cycle 0’s **q1** while any identifier the new plan lacks survives from the earlier cycle. For the 47.2% of HotpotQA questions that re-planned, the pool over which these retrieval metrics are fused is therefore the last cycle’s results plus whatever stale entries the last plan did not overwrite, not the union of everything the question retrieved. Chapter 5 states the defect in full; here it means the agentic retrieval rows are a lower bound on what the question actually found and an exact description of what the Synthesizer was shown.

Supporting-fact F1 is the metric on which the trade is favourable, and it is the one this system is built to win. **agentic_full** reaches SP-F1 of 0.481 on HotpotQA against 0.412 for **hybrid_rerank**, 0.377 for **dense_only** and 0.330 for **bm25_only**, and 0.421 on 2WikiMultihopQA against 0.327, 0.314 and 0.273; both margins over the strongest retrieval baseline are significant under the paired bootstrap. Supporting-fact EM moves with it, from 0.016 to 0.088 and from 0.016 to 0.124, though those absolute values say plainly that recovering the gold evidence set *exactly* remains rare.

Against the two generative baselines the comparison needs more care than a single column can carry, and an earlier draft of this report got it wrong. **naive_rag** and **self_ask** were previously reported at a supporting-fact F1 of exactly 0.000 and the zero was described as structural — systems that emit no supporting facts at all. That was a scoring defect in the harness, not a property of the systems: both baselines record an evidence list on every question, but they cite it by passage identifier where the adapter keys it by evidence identifier, so no citation ever resolved and the intersection was empty. With the fallback corrected, the two baselines are scored on their whole evidence pool and reach the starred SP-F1 figures in the tables, 0.412 and 0.405 on HotpotQA and 0.327 and 0.318 on 2WikiMultihopQA. Those figures are printed beside the agentic ones but are not comparable with them, and the star says so. The agentic system is scored on the sentences it actually cited, a mean of 1.5 sentences per question on HotpotQA and 1.6 on 2WikiMultihopQA; the baselines are scored on everything they retrieved, a mean of 6.9 and 8.9. The SP-P and SP-R columns show the two protocols pulling in opposite directions: on HotpotQA the agentic system’s cited set has a precision of 0.654 and a recall of 0.406, where **self_ask**’s pool has a precision of 0.302 and a recall of 0.696, and on 2WikiMultihopQA the figures are 0.558 and 0.367 against 0.246 and 0.627. A single F1 over two protocols that trade precision against recall this differently does not rank the systems, and the tables place no significance mark between rows of different protocol. The one supporting-fact column that is scored the same way for every row is *Pool SP-R*, the recall of the whole evidence pool, and on it the agentic system leads: 0.793 against 0.706 for **hybrid_rerank** on HotpotQA and 0.684 against 0.622 on 2WikiMultihopQA, both significant. So the agentic system retrieves more of the gold evidence than any baseline and then cites a small, precise subset of it; its cited-subset SP-F1 is a property of that citation behaviour as much as of the retrieval, and is not the number on which to rank it against a pool-scored baseline.

What survives that correction is narrower than the categorical claim it replaces, and it is still the claim this system is built to support. Citation grounding — the fraction of claims with at least one citation that resolves to a real evidence sentence — is 0.960 on HotpotQA and 0.976 on 2WikiMultihopQA for **agentic_full** and 0.000 for every baseline, because no baseline citation resolves to anything (Tables 4.5 and 4.6). That difference is categorical. The agentic system’s citations name specific sentences that exist and that mostly are gold; the baselines’ citations name nothing that can be checked. The defensible claim from this grid is therefore about grounded and auditable answers rather than about accuracy: the system returns a somewhat worse answer

with a precise and machine-checkable account of itself, and on the harder dataset it also recovers more of the gold evidence than any retrieval baseline.

The cost columns price that account. **agentic_full** spends 4.26 model calls per question on HotpotQA and 3.80 on 2WikiMultihopQA against 1.24 and 1.46 for **self_ask**, issues 14.83 and 16.00 tool calls against 2.41 and 2.86, and the plan it finally selected has a mean depth of 1.71 and 1.90 over 1.80 and 2.26 sub-queries. The *Saved* column counts model calls that a deterministic rule replaced, and its definition has been tightened since an earlier draft of this report, which credited the system with 7.10 and 7.19 saved calls per question — more than it spent — and drew a conclusion from that. The earlier count credited the routing table and the alias-based entity linker on every sub-query, although with the shortcut enabled the model router is unreachable and with alias linking configured there is no model linker to skip, so neither ever replaced a call this configuration could have made; and it omitted the extraction ladder, which does replace a real model call on every sub-query it resolves deterministically. Under the corrected definition — a saved call is one that a configured model path would have issued and a rule pre-empted — **agentic_full** saves 2.60 calls per question on HotpotQA and 3.10 on 2WikiMultihopQA, against 4.26 and 3.80 spent, and Tables 4.5 and 4.6 print the per-rule decomposition so that any other definition can be applied to the same numbers. The gated verifier adjudication and deterministic extraction displace a real but minority share of the inference a fully model-driven version of the same design would spend; the claim that they displace more than the Planner, Synthesizer and Verifier consume between them is withdrawn. What remains true is that at 52 tokens per second every call not made is several seconds of wall clock, and that the count is measured rather than assumed.

Latency is the column that must be read with the most care, and the tables print it as a median with the maximum beside it rather than as a mean, for a reason the HotpotQA row makes plain. On 2WikiMultihopQA **agentic_full** costs a median of 35.4s per question against 13.2s for **self_ask** and 8.3s for **hybrid_rerank**: roughly two and three-quarter times the decomposing baseline and a little over four times the non-agentic one, in exchange for the attribution gain above. On HotpotQA the median is 28.3s, against 4.5s and 4.9s, and the maximum is 65744.3s — 18.3 hours, on question `5a7af32e55429931da12c99c`, inside a single Synthesizer call. No arrangement of four model calls consumes that budget; the host was almost certainly suspended with the call open, and the recorded elapsed time is the suspension. An earlier draft reported the HotpotQA mean, which was that one question divided by 250 and described nothing that happens per question; the median is the figure to quote, and the maximum is printed rather than trimmed so the outlier stays in view. The measurement also identifies a defect in the budget itself. The 300s wall-clock budget of §4.1 is cooperative: it is tested between DAG nodes and before a call is issued, never during one, so it cannot pre-empt a blocking call, and the NLI and reranker invocations carry no timeout at all. One question therefore ran to roughly 220 times its budget without any guard firing. Bounding a single tool invocation, rather than only the question, is a correction this measurement identified and which the present results do not include; Chapter 5 lists it as a limitation.

4.7 Ablation Study

Three ablations attribute the system’s behaviour to its parts (Tables 4.7 and 4.8). Removing the planner collapses the sub-query graph to the question itself: a single sub-query is routed, retrieved, synthesised and verified, and the Verifier can still send it back. That is a one-node agentic system, not the **hybrid_rerank** baseline, and the table’s own caption is precise about what the non-agentic floor requires — removing planning, synthesis and verification together. Removing the knowledge graph withdraws bridge-entity discovery while leaving decomposition intact. Removing the verifier disables the backward edge, leaving a system that plans and retrieves in multiple steps but never revisits its own answer; the difference between it and the

Table 4.7: Ablation study on **hotpotqa**. Deltas are measured against **agentic_full** ($\dagger$), so a *negative* Δ F1 means the removed component was helping. **Bold** marks an ablation significantly *better* than the full system and $\downarrow$ one significantly worse (paired bootstrap, 95% CI of the difference excluding zero); unmarked differences are not significant. Sample sizes are stated per row and are not assumed equal: **agentic_full** $n = 250$, **agentic_no_planner** $n = 250$, **agentic_no_kg** $n = 184^\ddagger$, **agentic_no_verifier** $n = 250$, **hybrid_rerank** $n = 250$, against a 250-question evaluation slice. **hybrid_rerank** is repeated from the main results as the non-agentic floor, since removing planning, synthesis and verification reduces the pipeline to it. Latency is the median per question, matching Table 4.5; the maximum and the wall-clock overruns are reported there. - across a whole row marks a configuration that has not been run. **hybrid_rerank** is retrieval-only: it emits a ranking and no answer, so the EM, F1, Δ F1 and p columns show - because there is no answer to score, not because the run is absent. Scoring the unattempted answer would report 0.000 and assert a failed attempt that never happened. $\ddagger$ **marks a preliminary row**: **agentic_no_kg** ($n = 184$) had not covered the full 250-question evaluation slice when this table was generated, so the point estimates are over the questions completed so far and every delta, p -value and significance mark against it is withheld – a paired test between a prefix and the full slice is not a paired test. These numbers will move; they are not final results.

System	n	EM	F1	Δ F1 vs. full [95% CI]	p	LLM calls	Latency (s) med.
agentic_full	250	0.432 $\dagger$	0.510 $\dagger$	-	-	4.26	28.3
agentic_no_planner	250	0.456	0.543	+0.033 [-0.017, +0.086]	0.186	1.35	9.7
agentic_no_kg	184 $\ddagger$	0.418	0.525	-	-	4.23	11.0
agentic_no_verifier	250	0.396 $\downarrow$	0.474 $\downarrow$	-0.037 [-0.065, -0.011]	0.008	2.06	16.5
hybrid_rerank (<i>non-agentic floor</i>)	250	-	-	-	-	0.00	4.9

Table 4.8: Ablation study on **twowiki**. Deltas are measured against **agentic_full** ($\dagger$), so a *negative* Δ F1 means the removed component was helping. **Bold** marks an ablation significantly *better* than the full system and $\downarrow$ one significantly worse (paired bootstrap, 95% CI of the difference excluding zero); unmarked differences are not significant. Every row covers the same frozen 250-question evaluation slice as the main results – no ablation was run on a reduced subsample (**agentic_full** $n = 250$, **hybrid_rerank** $n = 250$). **hybrid_rerank** is repeated from the main results as the non-agentic floor, since removing planning, synthesis and verification reduces the pipeline to it. Latency is the median per question, matching Table 4.6; the maximum and the wall-clock overruns are reported there. - across a whole row marks a configuration that has not been run. **hybrid_rerank** is retrieval-only: it emits a ranking and no answer, so the EM, F1, Δ F1 and p columns show - because there is no answer to score, not because the run is absent. Scoring the unattempted answer would report 0.000 and assert a failed attempt that never happened.

System	n	EM	F1	Δ F1 vs. full [95% CI]	p	LLM calls	Latency (s) med.
agentic_full	250	0.224 $\dagger$	0.267 $\dagger$	-	-	3.80	35.4
agentic_no_planner	-	-	-	-	-	-	-
agentic_no_kg	-	-	-	-	-	-	-
agentic_no_verifier	-	-	-	-	-	-	-
hybrid_rerank (<i>non-agentic floor</i>)	250	-	-	-	-	0.00	8.3

full system is the contribution of this project, measured rather than asserted.

The study is complete on HotpotQA for two of the three components and open elsewhere, and each row’s sample size is printed so that no reader has to take that on trust. **agentic_no_verifier** and **agentic_no_planner** have both covered all 250 HotpotQA questions, so their deltas are paired comparisons on the identical question set. **agentic_no_kg** on HotpotQA is still running and the three ablations on 2WikiMultihopQA are not started at the time of writing; the HotpotQA **agentic_no_kg** row is marked preliminary and carries no delta, the 2WikiMultihopQA rows render as -, and nothing below is concluded from either.

The verifier ablation is the measurement this project exists to make, and it lands the way the architecture predicted. With the backward edge removed and everything else held fixed, exact match on HotpotQA falls from 0.432 to 0.396 and F1 from 0.510 to 0.474; the paired difference is $\Delta F1 = -0.037$ with a 95% interval of $[-0.065, -0.011]$ and $p = 0.008$, and both answer columns carry the marker for significantly worse. The loop therefore pays for itself on this dataset in the only currency the ablation measures: the questions it sends back come back better often enough to move the aggregate. The price is printed beside it. The full system spends 4.26 model calls per question against 2.06 without the verifier, and a median of 28.3 s against 16.5 s, so the edge roughly doubles both cost columns for a gain of just under four F1 points. Read with the error analysis below, which finds the Verifier far more often too permissive than too strict, the result says the mechanism works when it fires and fires on a minority of the questions where it should. It is one dataset; the report’s own standard elsewhere is that one dataset does not settle a direction, and the 2WikiMultihopQA row is the one this section is waiting for.

The planner ablation is the surprise, and it is reported as one rather than smoothed. **agentic_no_planner** scores *above* the full system on HotpotQA, at an exact match of 0.456 and F1 of 0.543 against 0.432 and 0.510, with $\Delta F1 = +0.033$ $[-0.017, +0.086]$ and $p = 0.186$; the difference is not significant, and the honest statement is that decomposition did not measurably help answer accuracy on this dataset and may have hurt it. It did so at 1.35 model calls per question and a median of 9.7 s, about a third of the full system’s calls and wall clock. The agent-metrics table shows why the row is not the **hybrid_rerank** floor: the one-node system still synthesised and verified, grounded its citations at 0.936, and its Verifier triggered a re-plan on 18.8% of questions — every one of which was discarded as a near-duplicate of the identity plan already run, so the executed re-plan rate is 0.000 and the row measures the forward pass alone. Its supporting-fact F1 of 0.508 is the highest of any complete row. Two readings are consistent with this and the grid does not separate them. The first is that HotpotQA’s two-hop questions are within reach of one well-formed hybrid query reranked by a cross-encoder — Recall@10 of 0.876 for that baseline says as much — so the Planner’s narrow sub-queries dilute a ranking that was already good, exactly the mechanism the nDCG@10 rows exposed above. The second is that the Planner’s decompositions are simply worse than no decomposition for an 8B model that then has to compose an answer across them, which would put the fault in synthesis rather than in planning. Either way the result is a finding about the planner and not an embarrassment to hide: the component that the brief’s Table 1 lists first is the one whose contribution this grid cannot yet show, and 2WikiMultihopQA, whose chains are longer, is where the question will be decided.

The graph ablation is the one for which a null result was always plausible, since Chapter 3 describes the induced graph as a navigation aid rather than a knowledge base, and it is the one still running. Its HotpotQA row had not covered the full 250 questions when these tables were generated — the caption states how many it had — so it is marked preliminary and carries no delta; it is printed so the reader can see the run exists, and nothing is concluded from it here.

Table 4.9: Failure profile on `hotpotqa`. Labels are assigned post hoc by `eval/error_analysis.py` from the trace and the gold supporting facts – deterministically, first match wins – in a cascade from “the system never had a chance” to “it had everything it needed and still got it wrong”. `verifier_false_accept` and `verifier_false_reject` are counted again in the totals, independently of the cascade: the ordered label makes a false reject nearly unreachable, and it is the count that makes the Verifier→Planner loop falsifiable. Corpus titles were loaded, so the `decomposition_error` rule’s “gold facts exist in the corpus” clause is checked rather than assumed. A configuration with no run has no column. `bm25_only`, `dense_only` and `hybrid_rerank` emit no answer at all, so their *Correct* is structurally 0 and every question is counted as an error; the informative rows for them are the retrieval labels, not the accuracy. [‡] **marks a preliminary row**: `agentic_no_kg` ($n = 184$) had not covered the full 250-question evaluation slice when this table was generated, so the point estimates are over the questions completed so far and every delta, p -value and significance mark against it is withheld – a paired test between a prefix and the full slice is not a paired test. These numbers will move; they are not final results.

Label	bm25_only	dense_only	hybrid_rerank	naive_rag	self_ask	agentic_full	agentic_no_planner	agentic_no_kg [‡]	agentic_no_verifier
<i>Failure labels: count (share of all questions)</i>									
<code>parse_failure</code>	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	1 (0.005)	0 (0.000)
<code>budget_exhausted</code>	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)
<code>retrieval_miss</code>	4 (0.016)	5 (0.020)	0 (0.000)	0 (0.000)	0 (0.000)	3 (0.012)	1 (0.004)	3 (0.016)	3 (0.012)
<code>decomposition_error</code>	105 (0.420)	82 (0.328)	62 (0.248)	44 (0.176)	31 (0.124)	16 (0.064)	46 (0.184)	8 (0.043)	3 (0.012)
<code>bridge_link_failure</code>	104 (0.416)	115 (0.460)	138 (0.552)	55 (0.220)	69 (0.276)	17 (0.068)	63 (0.252)	11 (0.060)	20 (0.080)
<code>synthesis_error</code>	17 (0.068)	42 (0.168)	33 (0.132)	19 (0.076)	16 (0.064)	57 (0.228)	23 (0.092)	46 (0.250)	61 (0.244)
<code>verifier_false_accept</code>	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	33 (0.132)	0 (0.000)	25 (0.136)	0 (0.000)
<code>verifier_false_reject</code>	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)
<i>Totals</i>									
Questions scored	250	250	250	250	250	250	250	184	250
Correct	0	0	0	124	126	108	114	77	99
Accuracy	0.000	0.000	0.000	0.496	0.504	0.432	0.456	0.418	0.396
Errors	250	250	250	126	124	142	136	107	151
Errors with no rule firing	20	6	17	8	8	16	3	13	64
Verifier false accepts	0	0	0	0	0	92	74	72	0
Verifier false rejects	0	0	0	0	0	2	0	2	0
Recoverable (some cycle was right)	0	0	0	0	0	2	0	2	0

4.8 Error Analysis

Failures are categorised automatically from the traces against the gold supporting facts, first match winning: parse failure, budget exhaustion, retrieval miss, decomposition error, bridge-link failure, synthesis error, verifier false accept, and verifier false reject. The last of these — a discarded cycle whose answer was correct while the selected one is not — is what makes the backward edge falsifiable, and it is reported whatever it shows.

The taxonomy accounts for nearly all of what went wrong, as Tables 4.9 and 4.10 show. Of `agentic_full`’s 142 HotpotQA errors and 194 on 2WikiMultihopQA, only 16 and 23 fall through the cascade with no rule firing, so the profile is a description of the failures rather than a partition of a residue. Neither dataset records a single parse failure or budget exhaustion in any configuration, which retires two concerns the design spent effort on: the five-rung parse ladder and the reserved-call policy are doing their work, and nothing in these results is an artefact of malformed model output or of a question that ran out of allowance. The counterweight column is the important one for the central claim. Two questions on HotpotQA and five on 2WikiMultihopQA were *recoverable* — some cycle produced a correct answer that was not the one finally selected — and those same counts are the false rejects. The backward edge is therefore not discarding good answers at any material rate; its problem, as §4.10 develops, is the opposite one.

One record in the HotpotQA trace shows the shape of that opposite problem better than the aggregate does, and it is the same question that produced the 18-hour latency above, `5a7af32e55429931da12c99c`. The system answered *New York City* where the gold answer is *Brooklyn, New York*, and the answer sentence it emitted alongside was copied from an evidence sentence unrelated to the question. The trace record scores that sentence at an NLI support

Table 4.10: Failure profile on `twowiki`. Labels are assigned post hoc by `eval/error_analysis.py` from the trace and the gold supporting facts – deterministically, first match wins – in a cascade from “the system never had a chance” to “it had everything it needed and still got it wrong”. `verifier_false_accept` and `verifier_false_reject` are counted again in the totals, independently of the cascade: the ordered label makes a false reject nearly unreachable, and it is the count that makes the Verifier→Planner loop falsifiable. Corpus titles were loaded, so the `decomposition_error` rule’s “gold facts exist in the corpus” clause is checked rather than assumed. A configuration with no run has no column. `bm25_only`, `dense_only` and `hybrid_rerank` emit no answer at all, so their *Correct* is structurally 0 and every question is counted as an error; the informative rows for them are the retrieval labels, not the accuracy.

Label	bm25_only	dense_only	hybrid_rerank	naive_rag	self_ask	agentic_full
<i>Failure labels: count (share of all questions)</i>						
<code>parse_failure</code>	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)
<code>budget_exhausted</code>	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)
<code>retrieval_miss</code>	4 (0.016)	0 (0.000)	1 (0.004)	1 (0.004)	1 (0.004)	1 (0.004)
<code>decomposition_error</code>	147 (0.588)	144 (0.576)	138 (0.552)	124 (0.496)	48 (0.192)	22 (0.088)
<code>bridge_link_failure</code>	45 (0.180)	49 (0.196)	50 (0.200)	42 (0.168)	81 (0.324)	7 (0.028)
<code>synthesis_error</code>	36 (0.144)	45 (0.180)	46 (0.184)	34 (0.136)	19 (0.076)	59 (0.236)
<code>verifier_false_accept</code>	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	82 (0.328)
<code>verifier_false_reject</code>	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)	0 (0.000)
<i>Totals</i>						
Questions scored	250	250	250	250	250	250
Correct	0	0	0	38	66	56
Accuracy	0.000	0.000	0.000	0.152	0.264	0.224
Errors	250	250	250	212	184	194
Errors with no rule firing	18	12	15	11	35	23
Verifier false accepts	0	0	0	0	0	154
Verifier false rejects	0	0	0	0	0	5
Recoverable (some cycle was right)	0	0	0	0	0	5

above 0.99 and a citation grounding of 1.0 — the sentence is, after all, entailed by the evidence it was copied from — against a retrieval agreement of 0.0, and the Verifier accepted the answer at a confidence of 0.82; Chapter efch:conclusions quotes the exact component values from the record. Nothing in the blend is miscomputed. The Verifier entails the `answer_sentence` field and never the `answer` itself, so an answer sentence that is faithful to a cited sentence but not to the question passes at high confidence regardless of what the answer string says. This is a property of the design rather than of the run, it is the single most instructive failure in the grid, and Chapter 5 carries it as a limitation.

One failure mode found during development deserves recording, because it is a property of agentic systems generally and not of this implementation. The verifier’s confidence blend weights entailment at 0.45. When the NLI encoder was unavailable, the verifier fell back as designed to token containment, which scored 1.000 support on an answer where the entailment model scores 0.0008 — correctly refusing to entail a comparison from one operand’s date alone. The fallback is safe in that it never raises; it is not safe for the results. Because entailment dominates the blend, a silent fallback pushes nearly every answer above the acceptance threshold, the verdict becomes uniformly *accept*, and the backward edge never fires. The run completes, the numbers look plausible, and the central measurement of the project quietly becomes an artefact.

Nothing in the output distinguishes that run from a healthy one except a degradation flag in the trace. The system now performs an explicit preflight check that loads the encoder and verifies a self-entailment before an evaluation begins, and refuses to start otherwise. The general lesson is that a graceful degradation path in an agentic system is a measurement hazard: it converts a loud failure into a quiet bias, and the component most worth protecting is the one carrying the most weight in a decision.

4.9 Verifier Threshold Calibration

Table 4.11: Post-hoc threshold sweep for the Verifier’s confidence blend on the hotpotqa calibration slice (50 questions, disjoint from the evaluation 250). † configured; * Youden-optimal; ‡ F1-optimal. **This sweep is post-hoc:** every confidence was produced by a single run at threshold 0.55, and changing the threshold would change whether a re-plan fires and therefore the answers themselves. It shows where the accept/reject boundary best separates correct from incorrect answers *given the confidences this system produced*; it does not predict the re-plan rate or the accuracy of a run executed at a different threshold, which would require one run per threshold. The re-plan column is a counterfactual read of the first-cycle confidences. Recommendation: KEEP 0.55. The *J*-optimal point on this slice is 0.54, 95% interval [0.45, 0.64] over 1000 bootstrap resamples of the 50 calibration questions; that interval is too wide, and the sample too small, to justify replacing a configured value.

Threshold	n_{acc}	Acc. rate	EM (acc.)	F1 (acc.)	EM (rej.)	Re-plan rate	Prec.	Rec.	Youden <i>J</i>
0.40	37	0.74	0.432	0.485	0.077	0.34	0.432	0.941	0.305
0.45	36	0.72	0.444	0.478	0.071	0.36	0.444	0.941	0.335
0.49‡	35	0.70	0.457	0.491	0.067	0.36	0.457	0.941	0.365
0.50	33	0.66	0.455	0.467	0.118	0.38	0.455	0.882	0.337
0.54*	32	0.64	0.469	0.481	0.111	0.42	0.469	0.882	0.367
0.55†	30	0.60	0.467	0.480	0.150	0.48	0.467	0.824	0.339
0.60	25	0.50	0.440	0.456	0.240	0.54	0.440	0.647	0.223
0.65	25	0.50	0.440	0.456	0.240	0.54	0.440	0.647	0.223
0.70	25	0.50	0.440	0.456	0.240	0.54	0.440	0.647	0.223
0.75	24	0.48	0.417	0.433	0.269	0.56	0.417	0.588	0.164

Tables 4.11 and 4.12 are generated by `calibrate.py` from the run

Table 4.12: Reliability of the Verifier’s confidence blend on the hotpotqa calibration slice: ECE = 0.332 over 10 equal-width bins, MCE = 0.636, Brier = 0.339 against a base-rate baseline of 0.224 (skill -0.510). The signal is *not* well calibrated as a probability, so the threshold must be read as a ranking cut-point rather than as a probability of correctness. Discrimination is a separate question from calibration: AUC = 0.633, i.e. the confidence ranks a correct answer above an incorrect one that often. The ordering is informative even though the scale is not, so a threshold on it is meaningful as a cut-point. An empty bin renders as a zero count and a - accuracy, never as an accuracy of zero.

Confidence bin	<i>n</i>	Mean conf.	Accuracy
[0.0, 0.1)	2	0.029	0.000
[0.1, 0.2)	0	—	—
[0.2, 0.3)	3	0.295	0.333
[0.3, 0.4)	8	0.359	0.000
[0.4, 0.5)	4	0.465	0.250
[0.5, 0.6)	8	0.549	0.500
[0.6, 0.7)	0	—	—
[0.7, 0.8)	1	0.721	1.000
[0.8, 0.9)	9	0.829	0.556
[0.9, 1.0)	15	0.969	0.333

agentic_full_hotpotqa_20260904T184256Z, the only run in the grid over the 50-question HotpotQA calibration slice of §2.6, which shares no question with the evaluation 250. They are placed here because they are results, but they are read in Chapter 5, where the threshold is discussed as a limitation: the sweep is post-hoc over confidences that one run at 0.55 produced, so it reports how well the confidence signal separates correct from incorrect answers and where the boundary sits on that signal, not what a run at a different threshold would have done. Every calibration figure quoted in that discussion is a cell of these two tables.

4.10 Discussion

The grid does not support the claim this project would have preferred to make. Agency did not buy accuracy. On both datasets a published decomposition method with no verification and no feedback edge answers more questions correctly than the four-agent system, at roughly a third of the model calls and a fraction of the wall clock, and the fused ranking the agentic system produces is significantly worse than a single hybrid query reranked by a cross-encoder. Any reading of these tables that concludes otherwise is reading them wrongly.

Agency did buy something else, and the failure profile shows what. Against **hybrid_rerank**, **agentic_full** reduces the share of questions labelled **decomposition_error** from 0.248 to 0.064 on HotpotQA and from 0.552 to 0.088 on 2WikiMultihopQA, and **bridge_link_failure** from 0.552 to 0.068 and from 0.200 to 0.028. Those are the two failure classes that Chapter 1 argued a single-shot ranker cannot address in principle, and they have very largely gone. What has replaced them is **synthesis_error**, at 0.228 and 0.236 — by a wide margin the dominant agentic failure mode, and more than three times the corresponding rate for **self_ask**, which is 0.064 and 0.076. The diagnosis this licenses is precise and it reconciles every result above. The system now finds the right evidence and then composes the wrong string from it. That is exactly the profile of a system with high supporting-fact F1 and low exact match, and it says that the marginal engineering effort belongs in extraction and synthesis rather than in more planning, more traversal, or more retrieval.

The verifier’s behaviour is the second half of the same picture. The traces record 92 false accepts on HotpotQA and 154 on 2WikiMultihopQA — answers the Verifier passed that the gold

supporting facts do not support — against 2 and 5 false rejects. The loop is therefore not too strict but far too permissive, and the threshold calibration reported in Chapter 5 explains why it cannot easily be made stricter: the confidence blend separates correct from incorrect answers only weakly, so moving the threshold trades false accepts for false rejects along a shallow curve rather than cleanly removing them. That places the ceiling on this design where the error profile already put it. A verifier that scores an answer sentence against its citations can detect that an answer is unsupported; it has no mechanism for detecting that a *well-supported* answer is the wrong span of a correct sentence, which is what a synthesis error usually is.

A practitioner reading this should take the following. If the deliverable is a ranked list of passages, use **hybrid_rerank**: it is the best ranker here, costs no model calls, and nothing in this report displaces it. If the deliverable is a short answer string and cost matters, use **self_ask**; it is the accuracy leader on both benchmarks at 1.24 and 1.46 model calls per question. The four-agent system earns its cost only where the answer must be attributable — where a reader has to be able to check which sentences support the claim, and where an unsupported answer is worse than a slow one. On that axis the difference is categorical in one respect and only in that one: the agentic system’s citations resolve to real sentences at a grounding of 0.960 and 0.976, and no baseline’s citations resolve at all. Its supporting-fact F1 is a property of a precise cited subset, and the pool-scored figures for **self_ask** and **naive_rag** are not the same measurement, so no ranking on that column is claimed. Whether an auditable answer is worth close to three times the median latency of the decomposing baseline is a property of the deployment and not of the benchmark, and the tables are reported in a form that lets the reader decide it rather than being told.

Two further observations hold independently of the ablation grid.

The first concerns where intelligence is worth spending, and what the run showed is not quite what was designed. The retrieval agent was built to route every sub-query through a seven-rule decision table and issue no model calls; it issued none, but the trace shows that the first rule — honour the Planner’s tool hint — fired on every one of the routing decisions in the headline HotpotQA run, because the Planner attached a hint to every sub-query it emitted, and the six lexical rules behind it never executed. The routing decision was therefore made by the model after all, inside the planning call the system pays for anyway, and the table’s contribution was to avoid asking twice. That is still a saving, and the graph navigator remains fully deterministic; but the finding is about the division of labour between agents rather than about rules replacing a model. It also closes off the experiment §3.3.2 designed to test the rules against a model router: with a hint on every sub-query the model router is unreachable as the code stands, and no agreement rate is reported. The open question this leaves is whether the Planner’s hints agree with what the lexical rules would have chosen, which the trace can answer offline and this report does not.

The second concerns self-report. The planner produces structurally correct decompositions — correct bridge shape, correct dependency placeholders — and then labels its own strategy wrongly on seven of eight questions, calling obvious bridge and comparison plans **single_hop**. The system therefore derives the strategy from the shape of the dependency graph and retains the model’s own label only for comparison. That an 8B model can build a correct structure and misname it is a specific, checkable claim about where such models can be trusted, and it generalises beyond this system: an agentic architecture that routes on a model’s self-assessment is resting on a weaker signal than one that routes on what the model actually produced.

Chapter 5

Conclusions and Future Work

5.1 Summary of Contributions

This project set out to test a narrow architectural claim: that adding one backward edge to a retrieval-augmented pipeline changes what the system is capable of, and that the change can be measured rather than merely described.

What was built is a four-agent system coordinated by an explicit state machine. The Planner decomposes a question into a validated directed acyclic graph of sub-queries connected by placeholders. The Retrieval Agent routes each sub-query to BM25, dense, or hybrid search via a seven-rule decision table over lexical features — of which, on the evaluated data, only the first rule ever fired, because the Planner attached a tool hint to every sub-query — fuses multiple formulations with reciprocal rank fusion [1], and reranks only when the fused margin is narrow. The Knowledge Graph Navigator links entities by alias matching and runs bidirectional breadth-first search over an entity graph induced from corpus passage text alone. The Verifier resolves citations, scores entailment with a DeBERTa cross-encoder, and — below its confidence threshold — returns a **ReplanDirective** to the Planner instead of an answer. The four agents cover all five tasks of Table 1 of the brief, and the system runs on one consumer GPU with no paid API.

Three properties are established independently of the benchmark outcome, because they were measured during construction. Structured planning with a local 8B model is reliable when the output is schema constrained: across the planning calls measured in development the model produced correctly shaped decompositions in 8 of 8 cases with zero format retries. The same model is unreliable at describing what it produced, mislabelling its own plan’s **strategy** field in 7 of those 8, which is why strategy is derived from the DAG shape and the model’s label discarded. And agency is affordable here only because most of it is deterministic: a full agentic question costs at worst 15 model calls against a cap of 20 and typically four to seven, since tool routing, entity linking, graph traversal, and the confidence arithmetic consume none.

5.2 Empirical Findings

Every number in this section is taken from the tables of Chapter 4, which are generated from the run artefacts; none is typed by hand, and where a run has not finished the finding is withheld rather than estimated. The headline is stated once, plainly, before the structure: the agentic system is not the most accurate system in this study, it is the only one that can show its work, and those two facts are the result.

The evaluation answers four questions. Each is stated explicitly because each has a defensible negative answer, and the protocol was fixed before any of them was resolved.

Does decomposition beat single-shot retrieval? Two reference points are needed, because no single baseline is comparable on every column. **hybrid_rerank**, the strongest non-

agentic ranker, is the reference for retrieval and supporting-fact quality; it attempts no answer, so it cannot be the reference for answer quality, and the generated tables render its answer columns as - rather than as a zero that would read as a failed attempt. The answer reference is **self_ask**, the strongest baseline that produces an answer string, and every answer-quality delta below is a paired bootstrap against it over the identical 250 questions. An earlier draft of this report described the planner’s fallback floor as **hybrid_rerank** “by construction” and argued that the agentic system could lose only through judgement; the ablation grid shows the floor is a one-node agentic system that still synthesises and verifies, and the argument is withdrawn. The answer is no, and it is a significant no on one of the two datasets. On HotpotQA **agentic_full** scores EM 0.432 and F1 0.510 where **self_ask** scores 0.504 and 0.603 and **naive_rag** 0.496 and 0.581; the paired difference in F1 against **self_ask** is -0.093 with a 95% interval of $[-0.145, -0.039]$, which excludes zero, so the deficit is established and not a fluctuation. On 2WikiMultihopQA the figures are 0.224 and 0.267 against **self_ask**’s 0.264 and 0.310 and **naive_rag**’s 0.152 and 0.200, and the paired difference of -0.043 $[-0.100, +0.014]$ spans zero, so there the deficit is a consistent point estimate and nothing stronger. Against single-shot retrieve-then-generate the picture is more favourable on the harder benchmark, where 2WikiMultihopQA’s longer chains punish a single query and **agentic_full** sits seven exact-match points above **naive_rag**, but no paired interval between those two is computed and their deltas against the common reference overlap, so that ordering is not certified either. What the grid does establish is that decomposition with verification loses to decomposition without it on the answer string, significantly on HotpotQA and directionally on 2WikiMultihopQA.

On evidence the answer is different in kind rather than simply reversed. Supporting-fact F1 is 0.481 against 0.412 for **hybrid_rerank** on HotpotQA and 0.421 against 0.327 on 2WikiMultihopQA, both significant under the paired bootstrap. The generative baselines were reported in an earlier draft at 0.000 on that metric, as systems that “emit no supporting facts”; that was a harness defect in resolving their citations, and corrected they score 0.405 and 0.412 on HotpotQA and 0.318 and 0.327 on 2WikiMultihopQA — starred in the tables, because they are scored on their whole evidence pool where the agentic system is scored on the sentences it cited. The two protocols trade precision against recall in opposite directions — on HotpotQA the agentic cited set scores a precision of 0.654 and a recall of 0.406 where **self_ask**’s pool scores 0.302 and 0.696 — and one F1 across them does not rank the systems. Scored the same way for every row, on the recall of the whole evidence pool, the agentic system leads every baseline significantly, 0.793 against 0.706 for **hybrid_rerank** on HotpotQA and 0.684 against 0.622 on 2WikiMultihopQA. What is categorical is citation grounding: 0.960 and 0.976 for the agentic system against 0.000 for every baseline, whose citations resolve to nothing. Fused retrieval quality moves the other way — nDCG@10 of 0.763 and 0.723 against 0.835 and 0.756 for the hybrid baseline, both significantly worse — because the reported ranking for a multi-hop system is the fusion of several narrow sub-query lists and fusion dilutes the head of the list. On 2WikiMultihopQA the two systems tie exactly on Recall@10 at 0.727, which locates the loss in ordering rather than in coverage. So decomposition improves which evidence is cited, and how checkably, and degrades where in a single fused list the evidence appears; the aggregate ranking metric alone would have concealed that.

Does the backward edge pay for itself? This is the central claim, and it is measured three ways. First, the marginal effect: **agentic_full** against **agentic_no_verifier**, which removes the loop and holds everything else constant. Second, conditional effectiveness, restricted to the questions where a re-plan fired and comparing the first cycle’s answer against the one finally selected — because if re-planning is rare the marginal effect will be small even when the mechanism works perfectly on the cases it reaches. Third, the counterweight: false rejects, where the Verifier sent back an answer that was already correct. A loop that fires often and improves nothing, or one that rarely fires, is a legitimate and reportable outcome; the trace records the selected cycle per question precisely so this can be settled from evidence. All three measurements

are now available on HotpotQA, and the decisive one answers the question the report was written to ask. The loop is genuinely exercised: the re-plan rate is 0.472 on HotpotQA and 0.392 on 2WikiMultihopQA, so on roughly two questions in five the Verifier declined to answer and sent a directive back to the Planner. Whatever else is true, the backward edge is not decorative, and the anti-repetition ban list on sub-query text is doing real work at that frequency. The counterweight is small: the traces record 2 false rejects on HotpotQA and 5 on 2WikiMultihopQA, and those are exactly the questions marked recoverable, meaning some discarded cycle had been right. Selecting the answer by maximum confidence over all cycles, with ties broken toward the earlier one, is therefore doing what it was designed to do — a bad re-plan almost never overwrites a good first attempt.

The failure lies in the other direction, and it is large. The Verifier accepted 92 answers on HotpotQA and 154 on 2WikiMultihopQA that the gold supporting facts do not support. A mechanism that fires on 47.2% of questions while rubber-stamping that many unsupported answers is not too aggressive but far too permissive, and the calibration evidence below says why raising the threshold is not an easy remedy.

The marginal effect is the central result of this report, and on HotpotQA it is positive and significant. **agentic_no_verifier** disables the backward edge and holds everything else fixed; over the same 250 questions it scores an exact match of 0.396 and F1 of 0.474 against the full system’s 0.432 and 0.510, a paired $\Delta F1$ of -0.037 with a 95% interval of $[-0.065, -0.011]$ and $p = 0.008$. The loop that fires on nearly half the questions does improve the aggregate, by just under four F1 points, at a cost of roughly doubling the model calls, 4.26 against 2.06, and the median latency, 28.3 s against 16.5 s. That is the claim the architecture was built to test, it is measured rather than asserted, and it lands in the direction the design predicted. It also has to be held at its true size. It is a single-dataset result, and the report’s own standard is that one dataset does not settle a direction; the 2WikiMultihopQA ablation has not been run and is the measurement that would make the finding a general one. And it is a statement about the edge, not about the system: a verifier that adds four points to an answer accuracy that is itself nine points behind **self_ask** has proved that feedback helps, not that this system is the one to deploy.

The planner ablation, complete on HotpotQA at the same time, produced the result the grid was not designed to expect. **agentic_no_planner** scores *above* the full system, at an exact match of 0.456 and F1 of 0.543, with $\Delta F1 = +0.033$ $[-0.017, +0.086]$ and $p = 0.186$ — not significant, but pointing against the component the brief lists first — at 1.35 model calls and a median of 9.7 s, roughly a third of the full system’s cost. It is not the **hybrid_rerank** floor: it still synthesised and verified, grounded its citations at 0.936, and its Verifier triggered a re-plan on 18.8% of questions, though every such re-plan was discarded as a duplicate of the identity plan and none executed. The reading that fits every other row is that HotpotQA’s two-hop questions are largely within reach of one well-formed hybrid query, so decomposition dilutes a ranking that was already good and asks an 8B model to compose across sub-answers it would not otherwise have had to. Whether the Planner earns its cost on the longer chains of 2WikiMultihopQA is the open question that dataset’s ablation row will answer, and the graph ablation on both datasets remains to be reported.

What does agency cost? Quality is reported alongside model calls per question, calls avoided by deterministic rules, tool invocations, plan depth, re-plan rate, and latency, so no accuracy gain is quoted without its compute price. On the same axis the tool-routing table is compared against model-driven selection on a held-out slice, by agreement rate, retrieval quality, and wall clock: the claim that a rule matches a model’s decision at zero cost is only worth making if the agreement rate is stated. It costs about three times the model calls of the strongest baseline and rather more than that in wall clock. **agentic_full** spends 4.26 model calls per question on HotpotQA and 3.80 on 2WikiMultihopQA against 1.24 and 1.46 for **self_ask**, and issues 14.83 and 16.00 tool calls against 2.41 and 2.86. Latency is reported as a median because one HotpotQA

question ran for 18.3 hours inside a single call, an outlier that an earlier draft’s mean reported as the per-question cost. On the medians, `agentic_full` takes 28.3 s per question on HotpotQA against 4.5 s for `self_ask` and 4.9 s for `hybrid_rerank`, and 35.4 s on 2WikiMultihopQA against 13.2 s and 8.3 s. Set against a result that is *worse* on answer accuracy, this is not a good trade for a system whose deliverable is an answer string, and the report says so rather than presenting the attribution gain as though it settled the arithmetic. It is a defensible trade only where an unattributable answer has little value.

The *Saved* column is the cost finding that had to be revised most. An earlier draft reported 7.10 and 7.19 model calls per question displaced by deterministic rules, more than the system spent, and called it the clearest engineering result in the work. The count was inflated: it credited a knowledge-graph entity-linking call on every sub-query that this configuration never issues, and it missed the extraction-ladder rungs that did avoid a call. Under the corrected definition, in which a saved call is one a configured model path would have made and a rule pre-empted, the column reads 2.60 and 3.10 calls per question against 4.26 and 3.80 spent, and Chapter 4 prints the per-rule decomposition beside it. The deterministic components displace a real minority of the inference a fully model-driven version of this design would spend; they do not displace more than the three reasoning agents consume, and that sentence is withdrawn wherever the earlier draft made it.

The comparison that would have tested the routing rules directly has not been run, and as the code stands cannot be. The rule-versus-model experiment described in Chapter 3 — clearing the `heuristic_shortcut` flag and re-running the calibration slice through the selection prompt — produced no artefact in `results/`, so no agreement rate is available and none is reported. The trace also shows why the experiment would not have measured what it was designed to measure: the Planner attached a tool hint to every sub-query in the headline run, the retriever honours a hint before it consults the shortcut flag, and so the model router is unreachable and the six lexical rules behind the hint rule never executed. The routing decision was made inside the planning call, and what the table saved was a second call for the same decision. That is a finding about where the division of labour between Planner and Retrieval Agent actually fell, and it should be read as one; the claim that lexical rules decide as a model would have decided is untested.

Where does it fail? The error analysis partitions failures by the stage that caused them — decomposition, first-hop retrieval, placeholder resolution, extraction, synthesis — because an aggregate score attributes nothing, and a system that fails mostly at retrieval calls for entirely different subsequent work than one that fails mostly at composition. It fails at composition, and it used to fail at retrieval. Measured against `hybrid_rerank`, the full system reduces the share of questions labelled `decomposition_error` from 0.248 to 0.064 on HotpotQA and from 0.552 to 0.088 on 2WikiMultihopQA, and `bridge_link_failure` from 0.552 to 0.068 and from 0.200 to 0.028. Those two categories are precisely the ones Chapter 1 argued a single-shot ranker cannot address in principle, and the machinery built to address them has very largely done so. Neither dataset records a single parse failure or budget exhaustion in any configuration, so nothing in these results is an artefact of malformed model output or of a question that ran out of allowance, and only 16 of 142 HotpotQA errors and 23 of 194 on 2WikiMultihopQA fall through the taxonomy unlabelled.

What is left is `synthesis_error`, at 0.228 and 0.236 — the dominant agentic failure mode on both datasets and more than three times the corresponding rate for `self_ask`, 0.064 and 0.076. Read together with a supporting-fact F1 that leads every retrieval baseline and an exact match that does not, the finding is specific: the system retrieves the right sentences and then extracts or composes the wrong span from them. This is the single most useful thing the evaluation produced, because it says where the next unit of engineering effort belongs. It does not belong in more planning — the planner ablation above says as much — nor in deeper traversal or better fusion, all of which are already delivering the evidence. It belongs in answer extraction, and by

extension in a verifier that can distinguish a well-cited answer from a correct one — a distinction the present confidence blend, which scores an answer *sentence* against its citations and never the answer string itself, is structurally unable to make. The limitations below give the trace record that demonstrates it.

One finding did not require the benchmark, because it surfaced while preparing to run it. It appears in the next section, being as much limitation as result.

5.3 Limitations

Sample size, and an incomplete grid. Results come from 250-question stratified samples of each dev split rather than the full splits. At that size the per-system 95% interval on token F1 spans roughly twelve points — $[0.451, 0.571]$ for **agentic_full** on HotpotQA — and an earlier draft argued from the overlap of such intervals that the deficit against **self_ask** could not be separated from noise. That was the wrong test: the systems answer the same questions, the comparison is paired, and the paired interval on the difference, $[-0.145, -0.039]$ on HotpotQA, excludes zero. The width of the marginal intervals is still the right caution about effect sizes — nothing here should be read as a precise one — but significance is decided by the paired column and not by overlap. The grid is also not finished. The ablations run against the same 250-question evaluation split rather than a subsample; at the time of writing the six baseline and full-system configurations are complete on both datasets, **agentic_no_verifier** and **agentic_no_planner** are complete on HotpotQA and not started on 2WikiMultihopQA, and **agentic_no_kg** is in flight on HotpotQA, short of 250 when the tables were last generated, and not started on 2WikiMultihopQA. The backward edge’s contribution is therefore a one-dataset number, the planner’s is a one-dataset null, and the graph’s is absent; each is marked so where it is due.

Model scale. Every result is conditional on a 4-bit quantised 8B model. The planner findings in particular are unlikely to transfer: a frontier model would probably label its own strategy correctly, making the DAG-derived override unnecessary, and would produce better decompositions in the first place, reducing the headroom the feedback loop has to work in. The cost profile is equally hardware-specific; at 52 tokens per second the case for replacing model calls with rules is far stronger than against a fast hosted endpoint.

Domain. Both benchmarks are built from Wikipedia: clean, well-segmented, densely hyperlinked, and written in a consistent register — and the entity graph exploits exactly those properties. Nothing in this work is evidence that it generalises to legal, biomedical, or enterprise collections, where entity mentions are less standardised and passages are longer and less self-contained.

Entity linking. Entities are linked by alias matching against a table built from passage titles and their mechanical variants, not by a trained linker. This is fast, deterministic, and interpretable, and it fails silently on nominal and pronominal reference and on any entity whose surface form in running text does not resemble its page title. Falling back to retrieved passage titles as seeds limits the damage but does not measure it.

An uncalibrated threshold. The Verifier’s confidence threshold of 0.55 and the weights of the confidence blend were set a priori, not learned. The threshold directly controls the re-plan rate, itself a headline agent metric, so tuning it on the evaluation questions would be leakage. The disjoint 50-question calibration slice of each dataset exists for exactly this purpose: the design called for the threshold to be swept over it and frozen before the evaluation run rather than accepted as a guess. That is not what happened. The sweep reported in Tables 4.11 and 4.12 was run after the evaluation, over the range $[0.40, 0.75]$ in steps of 0.01, on the 50-question HotpotQA calibration slice run **agentic_full_hotpotqa_20260904T184256Z**, reading confidences that a single run at 0.55 had already produced; the threshold that governed every evaluation run is the a priori one. The sweep returned a more useful answer than a better threshold. The confidence blend is not a probability. Expected calibration error is 0.332 and

maximum calibration error 0.636 against a base rate of 0.340, and the Brier score of 0.339 is *worse* than the constant predictor that always returns the base rate, a Brier skill of -0.510 . Nothing in this report should be read as saying that a confidence of 0.7 means a seven-in-ten chance of being correct, and the report does not say it anywhere. What the same signal does have is weak discrimination: the area under the ROC curve is 0.633, above chance but only modestly. Calibration and discrimination are different properties and this signal has the second without the first, which is exactly the profile of a usable *ranking cut-point* and an unusable probability. The threshold should be understood, and is used, as the former.

On the threshold itself the sweep declines to act, and the reasons matter more than the value. Youden’s J is maximised at 0.54, but a bootstrap over the selection returns a 95% interval of $[0.45, 0.64]$ — a width of 0.19 over a range swept from 0.40 to 0.75, which does not identify an operating point at all. The configured 0.55 lies inside that interval, and the paired comparison of decision accuracy between 0.54 and 0.55 gives $[-0.060, +0.060]$ at $p = 1.000$ with the selected value enjoying the advantage of having been fitted on the same 50 questions it was then scored on. Fifty labelled questions is below the threshold at which the calibration module will propose replacing a configured value at all, so its recommendation to retain 0.55 is a refusal to act on thin evidence rather than an endorsement of the guess. The honest statement is that the threshold remains unvalidated and that the study now knows precisely how unvalidated it is. A genuine sweep would require one full evaluation run per candidate threshold, since changing the threshold changes whether a re-plan fires and therefore changes the plan, the evidence and the answer; the post-hoc sweep reads confidences that were all produced under 0.55 and cannot substitute for that.

This connects directly to the Verifier’s 92 and 154 false accepts. With discrimination this weak, moving the acceptance boundary trades false accepts for false rejects along a shallow curve rather than removing them, so the permissiveness reported in the previous section is a property of the confidence signal and not of where the line was drawn on it. Improving the loop means improving the signal — atomic claim decomposition, cross-source agreement, a better entailment model — not retuning the threshold.

The Verifier checks the sentence, not the answer. The NLI hypothesis is the Synthesizer’s `answer_sentence` field, chosen because a declarative sentence is what an entailment model is trained on; the `answer` string that the metrics score is never entailed against anything. The two can come apart, and when they do the blend cannot see it. HotpotQA question `5a7af32e55429931da12c99c` is the cleanest instance in the trace. The system answered *New York City* where the gold answer is *Brooklyn, New York*, and emitted as its answer sentence a sentence copied from an evidence passage unrelated to the question. The Verifier scored NLI support at 0.995 and citation grounding at 1.0, since the copied sentence is trivially entailed by the sentence it was copied from, recorded a retrieval agreement of 0.0, and accepted the answer at a confidence of 0.82. Every component signal was computed correctly; the architecture asked the wrong question. A verifier that accepts an answer because its *explanation* is well supported will accept any answer whose explanation is a quotation, and that is a structural gap rather than a tuning one. The correction — entail a hypothesis built from the question and the answer string, or check that the answer string occurs in the entailed sentence — is straightforward, and it was not in place for any run reported here.

Re-planning overwrites the previous cycle’s state. Sub-query identifiers restart at `q1` in every plan revision, and the per-sub-query result, knowledge-graph result and extracted-answer maps on the question state are keyed by that identifier. A re-plan’s `q1` therefore overwrites cycle 0’s `q1`, while any identifier the new plan does not reuse survives from the earlier cycle, stale. Of the 118 HotpotQA questions on which `agentic_full` re-planned, 115 lost their cycle-0 retrieval results in this way and 92 carry at least one stale identifier from a plan that was no longer current. Two consequences follow. The retrieval metrics reported for the 47.2% of HotpotQA questions that re-planned are computed over a last-cycle-plus-stale pool rather than

over everything the question retrieved, so the agentic rows of the retrieval tables are a lower bound on what was found. And the trace is not the complete audit record the design document promises: the plans are preserved immutably, but the per-sub-query results they produced are not, so a reader cannot reconstruct what the Synthesizer was shown in cycle 0 of a re-planned question. The fix is to namespace the state by cycle; it is recorded as a known defect in the design document and was not applied to any run reported here.

The wall-clock budget is cooperative. The 300s question budget is tested between DAG nodes and before a model call is issued, never during one, and the NLI and reranker invocations carry no timeout at all. It therefore cannot pre-empt a call that blocks, which is how one HotpotQA question ran for 18.3 hours inside a single Synthesizer call — almost certainly a suspended host rather than a hung model — without any guard firing. The per-question latency tables print the median with that maximum beside it so the outlier stays visible; the defect stands, and a per-invocation bound is the correction.

Silent degradation, discovered the hard way. The Verifier never blocks: if the NLI cross-encoder cannot be loaded it falls back to token containment between the answer and the cited evidence. That fallback is safe for the process and catastrophic for the results. On a comparison claim supported by only one operand’s date, token containment scored 1.000 support where DeBERTa scores 0.0008 entailment on the same pair. Because NLI carries 0.45 of the confidence blend, a silent fallback pushes essentially every answer above the 0.55 threshold, every verdict becomes **accept**, and the re-plan loop — the mechanism the project exists to measure — never fires. The run completes, the numbers look plausible, and the central result is an artefact of a missing model file. A preflight check now confirms the encoder scores a self-entailment correctly before any evaluation starts. The general lesson outweighs the bug: graceful degradation and honest measurement are in direct tension, because the fallbacks that keep a long run alive are exactly the ones that quietly change what is measured. Every degradation path needs a preflight gate or a trace flag.

5.4 Future Work

Multi-agent coordination beyond a fixed topology. The agent graph here is static: the orchestrator invokes the same four agents in the same order, and the only dynamism is the re-plan edge. A natural extension instantiates agents per question — several retrieval agents specialising in different sub-corpora, or a negotiation between graph-first and text-first strategies whose disagreement is itself a signal. Workflow-level agent composition has a precedent in the geospatial copilot literature [4], and how many agents such a scheme can add before cost outruns benefit is directly measurable with the instrumentation already in place.

Learned rather than heuristic decision-making. The tool-routing table, the rerank gate, and the confidence blend are hand-specified functions whose inputs are logged on every question, making each a supervised learning problem with a free training set. The interesting version is not replacing the rules with a classifier but quantifying the gap: if a learned router beats the rule table by a negligible margin, that is a stronger result for the rules than any argument. The first step is cheaper than learning anything: the trace records the Planner’s hint and the lexical features on every sub-query, so the agreement between the hint that was honoured and the rule that would have fired can be computed offline, and it would say whether six of the seven rules were redundant or merely unreachable.

Stronger fact checking. The Verifier checks whether the final answer sentence is entailed by its citations. It does not check intermediate hops, does not cross-reference independent sources, and cannot distinguish an unsupported claim from one whose support was simply not retrieved. A verifier that decomposed the answer into atomic claims, verified each independently, and located contradictions between sources would give the re-plan directive far more precise information about what to look for next — and directive quality, not the loop’s existence, is plausibly what

limits its usefulness.

Domain transfer. The brief suggests legal and biomedical retrieval, and each stresses a different part. Biomedical corpora have curated ontologies, so the KG Navigator could traverse an authored graph instead of an induced one, isolating how much of the graph’s value comes from structure rather than from the induction procedure. Legal corpora invert the challenge: citation networks are explicit and reliable, but passages are long, densely cross-referential, and full of terms of art whose meaning is document-specific, testing the alias-matching linker to destruction.

5.5 Toward a Thesis Extension

Two of the thesis directions in the assignment brief follow from this system rather than requiring a new one. Both would also be the natural place to introduce the real-time external tools the brief describes and this system deliberately omits: the registry accepts a connector as one more entry, and the budgeting discipline applied here to model calls is what such a connector would need.

The self-reflective fact-checking track is the closer fit. The Verifier already scores confidence and triggers further retrieval when it is low, which is the mechanism the brief describes; what it lacks is multi-source cross-verification, provenance annotation, and calibrated uncertainty reported to the user rather than consumed internally. Extending it to a multimodal setting — validating retrieved imagery against textual reports, as the brief’s flood-extent example illustrates — needs a cross-modal entailment model in place of the NLI stage, but leaves the orchestration, the directive schema, and the anti-loop machinery intact.

The agentic query-decomposition track is the other. This Planner decomposes a static text corpus into a graph of at most five nodes; the spatio-temporal version decomposes over archives indexed by time and location, where sub-queries carry temporal extents and dependencies encode change detection between slices. The structure — validated DAG, placeholder substitution, per-node tool routing, verification with feedback — is unchanged; the tool registry and the cost model are not. The cost model is where the work sits: when one tool invocation is a satellite-imagery request rather than a BM25 query, the budgeting discipline applied here to language-model calls becomes the central design problem rather than an implementation detail.

Bibliography

- [1] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of SIGIR*, 2009.
- [2] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of COLING*, 2020.
- [3] Jakub Lala et al. Paperqa: Retrieval-augmented generative agent for scientific research. *arXiv preprint arXiv:2312.07559*, 2023.
- [4] Chachong Lee et al. Multi-agent geospatial copilots for remote sensing workflows. *arXiv preprint arXiv:2501.16254*, 2025.
- [5] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of EMNLP*, 2023.
- [6] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [7] Florian Schneider et al. Collex: A multimodal agentic rag system enabling interactive exploration of scientific collections. *arXiv preprint arXiv:2504.07643*, 2025.
- [8] Nandan Thakur, Nils Reimers, Andreas Ruckle, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- [9] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of ACL*, 2023.
- [10] Junde Wu, Jiayuan Zhu, and Yuyuan Liu. Agentic reasoning: Reasoning llms with tools for the deep research. *arXiv preprint arXiv:2502.04644*, 2025.
- [11] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-pack: Packed resources for general chinese embeddings. *arXiv preprint arXiv:2309.07597*, 2023.
- [12] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.
- [13] Ruichen Zhang et al. Toward agentic ai: Generative information retrieval inspired intelligent communications and networking. *arXiv preprint arXiv:2502.16866*, 2025.

- [14] Yongfeng Zhang et al. Ai agent for information retrieval: Generating and ranking. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, pages 5605–5607, 2024.